



VELS





INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI

LLM Probe: An Activation-Based Detection of Memorization and Hallucination in Large Language Models

FINAL YEAR

PROJECT PHASE-I REPORT

Submitted by

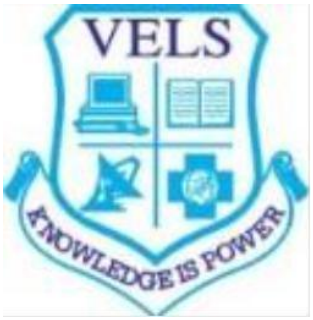
Jayachandran.K (23619211)

Praveen.R (23619225)

Mohamed Zipress.Z (23619221)

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE -
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project "LLM Probe: Activation-Based Detection of Memorization and Hallucination in Large Language Models" is the bonafide work of **Jayachandran.K (23619211), Praveen.R (23619225), Mohamed Zipress.Z (23619221)** who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. Ulaga Priya, M.E., Ph.D.
Associate Professor & Head
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

SIGNATURE OF THE SUPERVISOR
Mrs. F. Benasir Begam, M.E., (Ph.D.)
Assistant Professor
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

Submitted for the Project Work and Viva-Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support I received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of lab facilities have been a motivating factor in this completion of project work, all because of FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH., with great pleasure that I take this opportunity to thank him. I am grateful to our HEAD OF THE DEPARTMENT, Dr. K. Ulagapriya., for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. I extend our warmest thanks to my INTERNAL GUIDE, Mrs. F. Benasir Begam, who has patiently guided and provided us with invaluable advice and help. I am thankful to my Project Coordinator Ms. S. M. Sowmiya. I would like to express my greatest gratitude to our Staff members, Librarian, and Non-teaching staff members of Computer Centre, who have helped and supported me throughout. Last but not the Least, I wish to thank my parents and my friends for their individual support and interest who has inspired me and encouraged me to go in my own way, without them I would be unable to complete this project.

JAYACHANDRAN K	**23619211**
PRAVEEN R	**23619225**
MOHAMED ZIPRESS Z	**23619221**

TABLE OF CONTENTS

ABSTRACT **7**
LIST OF FIGURES **8**
LIST OF TABLES **9**
LIST OF ABBREVIATIONS **10**
INTRODUCTION **11**
1.1 General 11
1.2 Problem Description 11
1.3 Existing System 12
1.4 Proposed System 12
1.5 Objectives 13
1.6 Scope of the Project 14
1.7 Contributions 14
1.8 Modules 15
1.9 Summary 15
LITERATURE REVIEW **16**
2.1 Memorization in Language Models 16
2.2 Hallucination in Language Models 16
2.3 Extrinsic Hallucination Detection 16
2.4 Probing Classifiers 17
2.5 Internal States and Truthfulness 17
2.6 Small Models and Quantization 17
2.7 Existing Systems and Limitations 18
2.8 Research Gap 18
2.9 Summary 18
SYSTEM DESIGN **20**
3.1 Existing System 20
3.2 Proposed System 20
3.3 System Architecture 20
3.4 Overall Workflow 21
3.5 System Modules 22
3.6 Data Flow 22
3.7 Hardware Requirements 23
3.8 Software Requirements 23
3.9 Summary 24
IMPLEMENTATION AND METHODOLOGY **25**
4.1 Model Loading and Infrastructure 25
4.2 Memorization Dataset Construction 25
4.3 Manual Review of Memorization Labels 26
4.4 Hallucination Dataset Construction 27
4.5 Hallucination Label Review 28
4.6 Activation Extraction 29
4.7 Layer Selection 29
4.8 Probe Training and Evaluation 30
4.9 Prefix Augmentation (K = 5) 31
4.10 Teacher-Forcing Verification 31
4.11 Reproducibility and Data Management 31

4.12 Summary 32
RESULTS AND DISCUSSION 33
5.1 Experimental Setup 33
5.2 Choice of Representation 33
5.3 Memorization Probe Results 33
5.4 Teacher-Forcing Verification Results 34
5.5 Dataset-Scaling Experiment 36
5.6 Hallucination Probe Results 36
5.7 Prefix-Augmented Probes 38
5.8 Labelling Error Analysis 39
5.9 Computational Performance 39
5.10 Functional Testing 40
5.11 Overall Discussion 40
5.12 Comparison with Existing Work 41
5.13 Summary 41
LIVE INFERENCE TOOL AND DEMONSTRATION 42
6.1 Overview 42
6.2 Live Inference Tool 42
6.3 Live Inference Demonstration 43
6.4 Sample Live Inference Output 48
6.5 Summary 50
CONCLUSION AND FUTURE WORK 51
7.1 Conclusion 51
7.2 Limitations 51
7.3 Future Scope 52
7.4 Summary 52
REFERENCES 53
APPENDIX 55
9.1 Dataset Record Formats 55
9.2 Project Directory Structure 55
9.3 Main Commands 56
SOURCE CODE 57
10.1 Overview of the Source Code 57
10.2 Command-Line Entry Point (cli.py) 57
10.3 Configuration (config.py) 57
10.4 Model Loading (models/loader.py) 58
10.5 Activation Extraction (core/activations.py) 58
10.6 Layer Selection (core/layer_selection.py) 60
10.7 Probe Training and Evaluation (core/probe.py) 61
10.8 Live Token-by-Token Inference (demo/inference.py) 62
CONFERENCE APPLICATION 66

LLM PROBE

An Activation-Based Detection of Memorization and Hallucination in Large Language Models

Authors: Jayachandran K · Praveen R · Mohamed Zipress Z **Project Guide:** F. Benasir Begam ME., (Ph.D.)

ABSTRACT

Large language models can recite memorized training text or state falsehoods with full confidence, and neither failure shows in the output. LLM Probe reads the hidden states of Phi-3-mini-4k-instruct, running in 4-bit on a single 6 GB GPU, to detect both. Layers are ranked by Cohen's d, and lightweight probes are trained on the three best. Memorization is detected at the final layer (d = 0.66) with an F1 of 0.75, a teacher-forcing test confirms that the probe tracks recitation itself, and the hallucination probe reaches an F1 of 0.69. A live command-line tool scores every generated token with both probes, with no external corpus, retrieval system or second model.

OBJECTIVES

- ✓ Build labelled memorization and hallucination datasets from the model's own outputs.
- ✓ Find which layers best separate each failure mode using Cohen's d.
- ✓ Train probes and test them against a majority baseline and held-out categories.
- ✓ Verify with teacher forcing that the memorization probe tracks recitation.
- ✓ Run both probes live, token by token, on a 6 GB GPU.
- ✓ Prepare a benchmarking tool for LLM developers.

METHODOLOGY

1. **Build datasets.** 1,727 memorization and 796 TruthfulQA examples.
2. **Extract activations.** All 33 layers, mean-pooled over the answer.
3. **Select layers.** Top three by Cohen's d, training split only.
4. **Train probes.** Logistic regression and MLP vs majority baseline.
5. **Evaluate.** Held-out categories, prefix augmentation, teacher forcing.
6. **Live CLI.** Score every token with both probes and report risk.

RESULTS & DISCUSSION

d = 0.66 Memorization signal

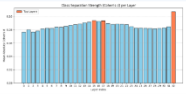
d = 0.11 Hallucination signal

F1 = 0.75 Memorization probe

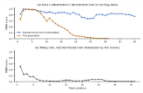
F1 = 0.69 Hallucination probe



Memorization. The final layer separates memorized from novel text most clearly. Selected layers: 32, 1, 0.



Hallucination. The signal is weaker and spread across middle layers. Selected layers: 32, 15, 17.



Teacher forcing. An Alice passage never seen in training keeps a high score (mean 0.845) when its true text is supplied.

Metric	Memorization probe	Hallucination probe
Test set	346	160
Majority-class baseline	91.34%	55.0%
Accuracy	95.91% (+4.5 pts)	66.9% (+11.9 pts)
Precision	0.79	0.72
Recall	0.71	0.66
F1 score	0.75	0.69

STANDARDS USED

- **IEEE 830 / ISO/IEC/IEEE 29148:** software requirements
- **IEEE 1016:** software design description
- **IEEE 829 / ISO/IEC/IEEE 29119:** software testing
- **ISO/IEC 25010:** software product quality
- **IEEE conference format:** paper and report
- **Evaluation:** ROUGE-L, TruthfulQA, Cohen's d, F1

CONCLUSIONS

Memorization and hallucination both leave a readable trace in a language model's hidden states. The memorization probe reaches an F1 of 0.75 and the hallucination probe 0.69, and teacher forcing confirms that the memorization probe tracks recitation. The hallucination probe beats the majority baseline by 11.9 points in-domain but is weaker on held-out categories. The probes need no corpus, retrieval system or second model, and they run live on a 6 GB GPU, so they work as a low-cost first-stage screen and a benchmarking tool for LLM developers.

ABSTRACT

Large language models can silently reproduce memorized training data or state falsehoods with full confidence, and neither failure shows up in the output text. We show both leave a measurable trace in the model's own hidden states, readable by a lightweight probe with no external corpus, retrieval system, or judge model. We build two parallel probing pipelines on Phi-3-mini-4k-instruct (3.8B parameters, 32 transformer layers, 3,072-dim hidden states), running entirely in 4-bit quantized form on a single 6 GB consumer GPU: a memorization probe trained on curated text excerpts, and a hallucination probe trained on TruthfulQA questions. Ranking layers by Cohen's d on the training split alone, we identify a strong memorization signal at the final layer ($d = 0.66$) and a weaker, more diffuse hallucination signal ($d = 0.11$). The memorization probe reaches an F1 of 0.75, improving 4.5 points over a 91.34% majority-class baseline to 95.91% accuracy. Teacher-forcing verification confirms the probe tracks genuine recitation: it scores 0.845 on the true continuation of a passage never seen in training and 0.000 on an unmemorized control passage, a clean separation with zero test-set leakage. Multi-round manual label curation on the hallucination side lifted in-domain accuracy from 58.7% to 66.9%, beating a 55.0% majority-class baseline, with the hallucination probe reaching an F1 of 0.69. Because both probes read activations the forward pass already computes, they add almost no inference-time cost; we integrate them into a live CLI tool that scores every generated token with both probes at once and reports a risk summary, a per-token score table, and risk charts for each response. Extraction runs at roughly 2.5 examples/second on consumer hardware, and probe training itself takes only seconds. Together, these results demonstrate that intrinsic activation probing is a practical, low-cost, fully offline strategy for flagging verbatim recitation, and a promising, extensible foundation for lightweight hallucination detection.

Keywords — large language models, interpretability, activation probing, hallucination detection, memorization, TruthfulQA, linear probes

LIST OF FIGURES

Figure No.	Title
Figure 3.1	System Architecture of LLM Probe
Figure 3.2	Data Flow Between System Processes and Data Stores
Figure 4.1	Hallucination Label Review Process
Figure 5.1	Cohen's d per Layer for the Memorization Probe (Training Split)
Figure 5.2	Per-Token Memorization Scores: (a) Alice in Wonderland, Teacher-Forced and Free Generation; (b) Moby-Dick, Teacher-Forced
Figure 5.3	Cohen's d per Layer for the Hallucination Probe (Training Split)
Figure 5.4	Hallucination Dataset Class Counts Across Review Passes
Figure 6.1	Flowchart of the Live Token-by-Token Inference Loop
Figure 6.2	Live Output for a Low-Risk Memorization Prompt (Colour of the Sun)
Figure 6.3	Live Output for a High-Risk Memorization Prompt (Binary Search Code)
Figure 6.4	Live Output for a Low-Risk Hallucination Question (Catching a Cold)
Figure 6.5	Live Output for a High-Risk Hallucination Question (Largest Peach-Producing State)
Figure 6.6	Live Output for a Low-Risk Memorization Prompt (Invented Planet Description)
Figure 6.7	Live Output for a High-Risk Memorization Prompt (Opening of Pride and Prejudice)
Figure 6.8	Live Output for a Low-Risk Hallucination Question (Wet Hair in Cold Weather)
Figure 6.9	Live Output for a High-Risk Hallucination Question (US Flag Question)

LIST OF TABLES

Table No.	Title
Table 1.1	Project Objectives
Table 3.1	Major System Modules
Table 3.2	Hardware Requirements
Table 3.3	Software Requirements
Table 4.1	Composition of the Memorization Dataset
Table 4.2	Memorization Labelling Rule
Table 4.3	Hallucination Label Review Passes
Table 5.1	Best-Layer Cohen's d by Pooling Strategy
Table 5.2	Memorization Probe In-Domain Performance
Table 5.3	Memorization Probe Confusion Matrix
Table 5.4	Teacher-Forcing Scores on Selected Passages
Table 5.5	Curated vs. Auto-Expanded Memorization Dataset
Table 5.6	Hallucination Probe Across Dataset Versions
Table 5.7	Hallucination Probe Confusion Matrix (In-Domain Test Split)
Table 5.8	Hallucination Probe In-Domain and Held-Out Results
Table 5.9	Standard vs. Prefix-Augmented Probes
Table 5.10	Representative Labelling Errors Corrected During Review
Table 5.11	Functional Testing Summary
Table 5.12	Comparison of LLM Probe with Existing Approaches
Table 6.1	Live Inference Test Prompts
Table 9.1	Memorization Dataset Fields
Table 9.2	Hallucination Dataset Fields

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
LLM	Large Language Model
GPU	Graphics Processing Unit
CUDA	Compute Unified Device Architecture
CLI	Command-Line Interface
MLP	Multilayer Perceptron
LR	Logistic Regression
ROUGE-L	Recall-Oriented Understudy for Gisting Evaluation (Longest common subsequence)
LCS	Longest Common Subsequence
TP / FP / FN / TN	True Positive / False Positive / False Negative / True Negative
KV cache	Key-Value cache
EOS	End of Sequence
QA	Question Answering
TruthfulQA	Truthfulness question-answering benchmark

CHAPTER 1
INTRODUCTION

1.1 General

Large language models are now part of search engines, programming assistants, tutoring systems, and customer-facing chat applications. In many of these settings, people act on what the model says without checking it. This makes the reliability of the generated text a practical concern and not only a research question.

Two failure modes stand out. The first is verbatim memorization, where the model reproduces a sequence from its training data word for word. When that data contains personal information, memorization becomes a privacy risk, and when it contains licensed text or code, it becomes a copyright risk [3], [12]. The second is hallucination, where the model produces a fluent, confident statement that is unsupported or false [15]. A hallucinated answer to a medical, legal, or financial question can mislead a user who has no reason to doubt it.

Both failures are hard to notice. A memorized paragraph reads exactly like a freshly written one, and a hallucinated fact is stated in the same tone as a correct fact. Nothing in the output text itself signals that something has gone wrong. Detecting these failures therefore requires information from outside the text.

Most current tools obtain that information from outside the model: they search a copy of the training corpus, retrieve documents from a knowledge base, or ask a second model to evaluate the first. This project takes a different route and looks inside the model. While a transformer generates text, each of its layers produces an internal vector for every token, called a hidden state. If reciting a memorized passage and inventing an answer involve different internal computations, those differences may be visible in the hidden states, and a small classifier, called a probe, could learn to recognise them.

1.2 Problem Description

The problem addressed by this project is how to detect verbatim memorization and hallucination in a language model's output without relying on an external corpus, a retrieval system, or a second model at inference time. The detection should work on a small model that runs on consumer hardware, it should add very little computation to generation, and it

should produce a score for each generated token so that a user can see which parts of an answer are at risk.

Several practical difficulties make this problem harder than it first appears. First, there is no ready-made labelled dataset that pairs a specific model's own outputs with memorization labels; such labels have to be created by comparing the model's continuations with known source text. Second, labelling hallucinations automatically is unreliable, because similarity metrics measure word overlap and not meaning: an answer that mentions a misconception in order to reject it can look similar to the misconception itself. Third, even if a probe performs well on the data it was trained on, it may have learned surface features of that data, so its behaviour on new kinds of text has to be tested separately. Finally, the whole pipeline had to fit within 6 GB of GPU memory, which constrained the model size and required careful checkpointing of long-running jobs.

1.3 Existing System

Existing approaches to both problems are mostly extrinsic. Memorization is usually measured by prompting a model with the beginning of a known document and checking whether its continuation matches the rest of that document, or by searching the training corpus for long overlapping n-grams [3], [12]. These methods need access to the training data or to a large reference corpus, and they are applied offline, after generation, as an audit.

Hallucination detection is commonly done by retrieving evidence from a trusted source and comparing it with the answer, by asking a second language model to judge the answer, or by sampling several answers to the same question and measuring how much they disagree [16], [17]. Retrieval requires an up-to-date knowledge base, judge models add the cost of a second model, and sampling-based methods require several generations for every question. All of them look only at what the model outputs.

Research on probing has shown that internal activations carry information about truthfulness [4], [5], [13], [14], but these studies usually focus on a single failure mode, use large models on data-centre hardware, and are not packaged as a tool that scores text while it is being generated.

1.4 Proposed System

LLM Probe is an intrinsic detection system that reads the hidden states of Phi-3-mini-4k-instruct while it generates text and scores each token for memorization and hallucination

risk. The system has two parallel pipelines that share the same core components. Each pipeline first builds a labelled dataset from the model's own generations, then extracts hidden-state activations for every labelled example, ranks all layers by how well they separate the two classes, and trains a lightweight probe on the most informative layers.

For memorization, the model is prompted with the opening of a known passage, and its greedy continuation is compared with the true continuation using ROUGE-L. For hallucination, the model answers every question in the TruthfulQA benchmark [1], and each answer is compared with the benchmark's lists of correct and incorrect reference answers. In both cases, cases that automatic scoring cannot decide are sent to manual review instead of being forced into a class.

The trained probes are stored together with the settings needed to reproduce their features, and are loaded by a command-line tool that runs both probes at once. As the model generates a response, the tool extracts the relevant activations for each new token and scores it with the memorization and hallucination probes. When generation ends, it prints the generated answer, a risk summary for each probe, a table of per-token scores coloured by risk level, and a chart of how each probe's score changed across the response.

1.5 Objectives

The objectives of this project are summarised in Table 1.1.

No.	Objective
1	Construct a labelled memorization dataset from Phi-3-mini's own continuations of known passages, with manual review of borderline cases.
2	Construct a labelled hallucination dataset from Phi-3-mini's answers to all 817 TruthfulQA questions, with a documented, auditable review process.
3	Extract hidden-state activations from all layers of the model and compare last-token and mean-pooled representations.
4	Identify the most informative layers for each task using Cohen's d effect size computed on the training split only.
5	Train and evaluate linear and MLP probes, comparing every result against a majority-class baseline and testing generalization on held-out categories.
6	Verify that the memorization probe responds to recitation itself using a teacher-forcing experiment.
7	Integrate the probes into a live, token-by-token inference tool that runs on a 6 GB consumer GPU.

Table 1.1: Project Objectives

1.6 Scope of the Project

The project covers one model, Phi-3-mini-4k-instruct, run in 4-bit quantized form. The memorization study uses English text from three kinds of sources: book openings from Project Gutenberg, lead paragraphs from Wikipedia, and algorithm implementations from Rosetta Code. The hallucination study uses the English TruthfulQA benchmark in its generation format. Probes are limited to logistic regression and a small multilayer perceptron trained on pooled activations of three selected layers.

The project does not attempt to explain which individual neurons or circuits cause memorization or hallucination, does not modify or fine-tune the model, and does not compare against larger model families. It also does not claim that a high probe score proves that a specific sentence was in the training data, or that a low hallucination score proves an answer is true. The probes provide risk signals that can help decide when a more expensive external check is worthwhile.

1.7 Contributions

The main contributions of this project are:

- Two labelled datasets built from a small model's own outputs, one for memorization and one for hallucination, together with the review records that document every label change.
- A shared, reusable pipeline for activation extraction, layer ranking by effect size, and probe training, applied identically to both failure modes.
- A direct comparison of the two failure modes under the same method, showing a strong memorization signal ($d = 0.61$) and a more diffuse hallucination signal ($d = 0.11$).
- A teacher-forcing experiment showing that the memorization probe responds to recitation and not to the familiarity of the prompt.
- A documented negative result on dataset scaling, showing how automated labelling rules can reduce probe quality if they are not audited.
- A command-line tool, with single-prompt and interactive modes, that scores every generated token with both probes at once and reports risk summaries, per-token scores, and risk charts on consumer hardware.

1.8 Modules

The system is organised into five modules. They are listed with their code components in Table 3.1 and implemented in Chapter 4.

- Dataset construction: collects source passages and TruthfulQA questions, generates the model's own continuations and answers, and labels them with ROUGE-L rules and manual review.
- Activation extraction: runs the model with hidden-state output and pools the activations of every layer over the generated span.
- Layer selection: ranks the layers by Cohen's d on the training split and keeps the three most informative ones.
- Probe training and evaluation: trains logistic-regression and MLP probes and compares them with a majority-class baseline and with held-out categories.
- Live inference tool: loads the stored probes and scores every generated token with both probes in a command-line session.

1.9 Summary

This chapter explained why verbatim memorization and hallucination matter, described how existing detection methods depend on external resources, and introduced LLM Probe, which reads the hidden states of Phi-3-mini-4k-instruct to detect both. It also set out the objectives, scope, contributions, and modules of the project. The next chapter reviews the literature on which the project builds.

CHAPTER 2
LITERATURE REVIEW

2.1 Memorization in Language Models

Carlini et al. [12] showed that individual training examples, including names, phone numbers, and addresses, can be extracted from GPT-2 by generating many samples and filtering them for text that the model is unusually confident about. Their work established memorization as a practical privacy risk and not only a theoretical one.

In follow-up work, Carlini et al. [3] measured memorization systematically across several model families. They prompted models with a prefix taken from the training data and counted how often the model reproduced the true continuation exactly. They found that memorization increases with model size, with the number of times an example is duplicated in the training data, and with the length of the prompt. Our memorization dataset uses the same prefix-and-continuation method at a much smaller scale.

2.2 Hallucination in Language Models

Ji et al. [15] survey hallucination across natural language generation tasks, distinguishing intrinsic hallucinations, which contradict the source or input, from extrinsic hallucinations, which add information that cannot be verified. In open-ended question answering, the most relevant form for this project is the confident statement of a false fact, often one that repeats a common misconception.

TruthfulQA, introduced by Lin et al. [1], was designed to measure this behaviour. It contains 817 questions across 38 categories, such as health, law, finance, and misconceptions, chosen so that a model imitating common human text is likely to give a false answer. Each question comes with a best answer, a list of correct answers, and a list of incorrect answers, which makes it possible to label a model's free-form answer by comparing it with both lists.

2.3 Extrinsic Hallucination Detection

Several detection methods work from outside the model. Farquhar et al. [16] sample multiple answers to the same question, group them by meaning, and compute the semantic entropy of the groups; high entropy indicates that the model is uncertain and more likely to confabulate. Kadavath et al. [17] show that language models can, to a useful degree,

estimate whether they know the answer to a question when asked to evaluate their own answers.

These methods are effective, but they require several generations per question or an additional self-evaluation step, which multiplies the cost of inference. Retrieval-based fact checking requires a reliable, current knowledge source. None of them provide a signal during a single, ordinary generation.

2.4 Probing Classifiers

A probing classifier is a simple model trained on the frozen internal representations of a larger network to test what information those representations contain. Alain and Bengio [6] introduced linear classifier probes as a way to understand what each layer of a deep network encodes. Belinkov [7] reviews the method in detail and points out an important caution: a probe can succeed by picking up surface correlates of a property instead of the property itself, so probe results need controls and careful interpretation.

Linear probes are preferred when the goal is to test whether information is present, because a linear classifier cannot construct complex features on its own. If a linear probe succeeds, the information must already be available in the representation in a readable form.

2.5 Internal States and Truthfulness

Azaria and Mitchell [4] trained classifiers on the hidden layers of language models to predict whether a statement the model is processing is true or false, and showed that internal states carry a truthfulness signal. Burns et al. [13] found directions in activation space that correspond to truth without using any labels, by searching for representations that are logically consistent across a statement and its negation.

Chen et al. [5] proposed INSIDE, which measures the consistency of internal states across several sampled generations to detect hallucination. Orgad et al. [14] showed that language models encode more information about their own errors than their outputs reveal, and that this information is concentrated in specific tokens and layers. Together, these studies motivate the use of hidden states as a detection signal.

2.6 Small Models and Quantization

Phi-3-mini [2] is a 3.8-billion-parameter model trained on heavily filtered web data and synthetic data, designed to run on local devices. Its small size makes it suitable for

experiments that require many forward passes on limited hardware, and its filtered training data raises the question of how much verbatim memorization remains.

Quantization reduces memory use by storing model weights with fewer bits. The 4-bit quantization scheme introduced with QLoRA [8] and implemented in the bitsandbytes library allows a model of this size to run within 6 GB of GPU memory while preserving most of its quality. The Hugging Face Transformers library [11] exposes the hidden states of every layer during a forward pass, which is essential for probing.

2.7 Existing Systems and Limitations

The literature leaves several practical gaps. Memorization studies are mostly large-scale audits that require access to the training corpus and are run after the fact. Hallucination detectors either depend on external knowledge or multiply the cost of generation. Probing studies show that the relevant information exists inside the model, but they usually study one failure mode at a time, use large models, and rarely report how well a probe transfers to categories of data it was not trained on.

Another gap concerns labelling. Probing results are only as good as the labels they are trained on, yet the process of creating labels, and the errors that automatic labelling introduces, is rarely described in detail. A probe trained on noisy labels can appear to work while learning the wrong thing.

2.8 Research Gap

Based on this review, the gap addressed by this project is the combination of four elements that are usually studied separately: probing for both memorization and hallucination with one shared method, on a small model that runs on consumer hardware, with labels created from the model's own outputs through a documented and audited process, and with explicit tests of cross-category generalization. The project also turns the trained probes into a working tool that scores text during generation, which shows how intrinsic detection could be used in practice.

2.9 Summary

This chapter reviewed work on memorization and hallucination in language models, on probing classifiers and internal states, and on running small quantized models. Existing methods either need external data or several generations, and probing studies rarely cover

both failure modes or report how a probe transfers to unseen categories. These gaps define the research gap addressed by this project. The next chapter presents the system design.

CHAPTER 3
SYSTEM DESIGN

3.1 Existing System

In existing workflows, checking a model's output for memorization or hallucination is a separate step that happens after generation. A developer who wants to know whether a response contains memorized text must search a large reference corpus, and a developer who wants to know whether a response is accurate must retrieve evidence or call a judge model. These steps are expensive, depend on resources that may not be available, and give no information about which part of a long response is at risk.

3.2 Proposed System

The proposed system moves detection inside the generation process. It is organised as a set of offline stages that build datasets and train probes, followed by an online stage that uses the trained probes during generation. The offline stages run once per dataset version, and their outputs are stored on disk. The online stage loads these stored outputs and adds only a small amount of computation per token.

Both probes share the same core library: the model loader, the activation extractor, the layer ranking module, and the probe training and evaluation module. Only the dataset construction scripts are specific to each task. This design ensures that the comparison between memorization and hallucination is fair, because both are measured with exactly the same method.

3.3 System Architecture

Figure 3.1 shows the overall architecture of the system. It can be read as four layers. The data layer contains the external sources: book openings, Wikipedia leads, and code for the memorization task, and the TruthfulQA benchmark for the hallucination task. The processing layer contains the two dataset builders, the activation extractor, the layer selection module, and the probe trainer, all of which use the same quantized model. The storage layer holds the labelled datasets, the activation cache, and the trained probe payload. The presentation layer is the command-line tool that uses the stored probe during live generation.

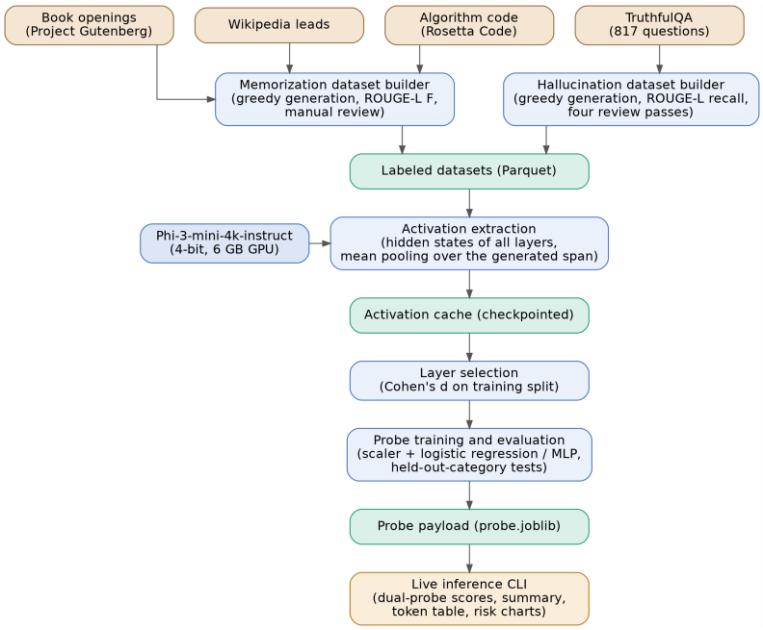


Figure 3.1: System Architecture of LLM Probe

Every stage reads its input from, and writes its output to, a file on disk. This makes each stage restartable and allows later stages to be rerun without repeating the expensive generation and extraction steps.

3.4 Overall Workflow

The workflow for building and using a probe proceeds in the following order:

1. Source text or benchmark questions are collected and stored as raw data.
2. The model generates a continuation or answer for every item using greedy decoding.
3. Each generation is scored against its reference text and labelled automatically where the score is decisive.
4. Borderline and ambiguous cases are exported to review files, decided by hand, and merged back with validation.
5. The model is run again on each labelled example to extract the hidden states of all layers, which are pooled and cached.
6. All layers are ranked by Cohen's d on the training split, and the top three are selected.
7. Probes are trained on the selected layers, evaluated in-domain and on held-out categories, and saved.

8. The command-line tool loads the saved probes, scores every token of a new generation with both probes, and reports the answer, a risk summary, a per-token table, and risk charts.

3.5 System Modules

Table 3.1 lists the main modules of the implementation and their responsibilities.

Module	Responsibility
config	Central settings such as the model name, paths, and layer count.
models/loader	Loads the 4-bit quantized model and its tokenizer.
core/activations	Runs forward passes with hidden-state output and pools activations over a chosen token span.
core/layer_selection	Computes Cohen's d for every layer and representation, ranks layers, and plots the ranking.
core/probe	Trains, evaluates, saves, and loads logistic-regression and MLP probes.
Memorization scripts	Collect source passages, build and label the memorization dataset, support manual review, and train the probe.
Hallucination scripts	Generate TruthfulQA answers, label them, run the review workflows, and train the probe.
demo/inference and render	Token-by-token generation with dual-probe scoring, and terminal output of the answer, summary, token table, and risk charts.
cli	Command-line entry point for single-prompt and interactive (REPL) sessions.

Table 3.1: Major System Modules

3.6 Data Flow

Figure 3.2 shows how data moves between the three main processes of the system and its two persistent stores. Raw sources are turned into labelled examples by the preparation process and stored as datasets. The training process reads the datasets and writes the probe payload. The live inference process reads the probe payload, receives a prompt from the user, and returns scored tokens.

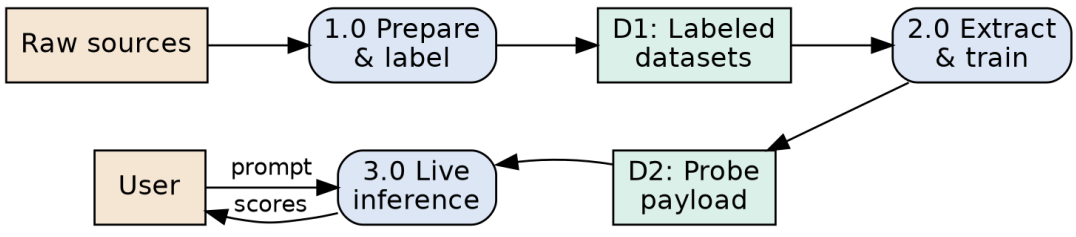


Figure 3.2: Data Flow Between System Processes and Data Stores

The datasets and the probe payload are the only interfaces between the offline and online parts of the system. The live tool never needs the raw data or the activation cache, which keeps it small and fast to start.

3.7 Hardware Requirements

Table 3.2 lists the hardware used for development and experiments.

Component	Specification / Purpose
GPU	Consumer NVIDIA GPU with 6 GB of memory, used for generation and activation extraction with CUDA.
Development computer	Windows machine used for dataset construction, training, and running the live tool.
Storage	Local disk for datasets, activation caches (several hundred megabytes), and trained probes.
Internet connection	Needed only to download the model, the TruthfulQA benchmark, and source passages.

Table 3.2: Hardware Requirements

The 6 GB memory limit shaped several design decisions. The model is loaded in 4-bit form, generation is done one example at a time, and all long-running jobs save their progress regularly so they can resume after an interruption.

3.8 Software Requirements

Table 3.3 lists the main software components.

Component	Purpose
Python	Implementation language for all components.
PyTorch [20]	Tensor computation and GPU execution.
Hugging Face Transformers [11]	Model and tokenizer loading, generation, and hidden-state output.
bitsandbytes	4-bit quantization of the model weights.
Hugging Face Datasets [21]	Loading the TruthfulQA benchmark.
rouge-score	ROUGE-L scoring for automatic labelling.
scikit-learn [10]	Logistic regression, MLP, standard scaling, data splitting, and metrics.
pandas, PyArrow, joblib	Dataset storage (Parquet, CSV) and saving the probe payload.
matplotlib, Git, GitHub	Result plots and version control for code, datasets, and results.

Table 3.3: Software Requirements

3.9 Summary

This chapter described the existing and proposed systems, the architecture, workflow, modules, and data flow of LLM Probe, and the hardware and software it needs. The design keeps dataset construction, probe training, and live inference separate, so the live tool needs only the saved probes and the model. The next chapter explains how each stage was implemented.

CHAPTER 4
IMPLEMENTATION AND METHODOLOGY

4.1 Model Loading and Infrastructure

All experiments use microsoft/Phi-3-mini-4k-instruct [2], loaded through the Hugging Face Transformers library [11] with 4-bit quantization from the bitsandbytes backend [8]. The model has 32 transformer layers and a hidden size of 3,072. Loading the model this way fits comfortably in 6 GB of GPU memory and leaves room for activations during forward passes.

All generation uses greedy decoding, meaning that the model always picks its single most likely next token. This makes every output deterministic: the same prompt always produces the same text, so every label in the datasets can be reproduced exactly, and the manual review decisions always refer to the same generated answer.

Long jobs, such as generating 817 answers or extracting activations for hundreds of examples, write their results to disk in small batches. When a job is restarted, it reads the existing results and skips the examples that are already complete. This was essential on a single consumer GPU, where a complete run could not always finish in one uninterrupted session.

4.2 Memorization Dataset Construction

Source passages were collected from three kinds of text that are likely to appear in web-scale training data: the opening paragraphs of well-known books from Project Gutenberg [22], the lead paragraphs of Wikipedia [24] articles, and canonical algorithm implementations from Rosetta Code [23]. Each passage was split into a prompt, made of its first 30 to 50 tokens, and a true continuation of up to 100 tokens. Table 4.1 shows the composition of the final dataset.

Source category	Memorized	Novel	Total
Wikipedia leads	15	877	892
Book openings	5	603	608
Code (Rosetta Code)	53	170	223
Other (lyrics, invented prompts)	0	4	4
Total	73	1,654	1,727

Table 4.1: Composition of the Memorization Dataset

The model received only the prompt and generated a continuation of the same length. The generated continuation y was compared with the true continuation c using ROUGE-L [18], which is based on the length of the longest common subsequence (LCS) between the two texts:

$$R = \mathrm{LCS}(y, c) / |c|, \quad P = \mathrm{LCS}(y, c) / |y| \tag{1}$$

$$F = 2 \cdot P \cdot R / (P + R)$$

Here R is the recall, the fraction of the true continuation covered by the generated text, and P is the precision, the fraction of the generated text that matches. Examples with an F-score above 0.7 were labelled memorized, and those below 0.5 were labelled novel. Examples between the two thresholds were not labelled automatically; they were exported for manual review. Examples below 0.5 whose first 10 or more tokens matched the true continuation exactly were also sent to review, since an exact opening match can indicate recall even when the rest of the text diverges. Table 4.2 summarises the labelling rule.

ROUGE-L F-score	**Label**
Above 0.7	Memorized
Between 0.5 and 0.7	Sent to manual review
Below 0.5, with 10 or more matching prefix tokens	Sent to manual review
Below 0.5, otherwise	Novel

Table 4.2: Memorization Labelling Rule

4.3 Manual Review of Memorization Labels

Automatic scoring cannot tell the difference between recalling a specific text and producing phrasing that is simply common. During manual review, each candidate was placed in one of three groups. The first group contained distinctive recall, such as an exact numeric constant, an unusual choice of algorithm reproduced token for token, or a rare phrase copied from the source. These were accepted as memorized. The second group contained matches on well-known facts. A model can state that Adam Smith wrote The Wealth of Nations without having memorized the specific article that says so, so these were not counted as memorization. The third group contained matches on generic phrasing, such as common connecting phrases or standard code idioms, which were also treated as novel.

The final memorization dataset contains 1,727 examples, of which 73 are memorized and 1,654 are novel (Table 4.1). The positive rate of about 4% is low, which is consistent with

the heavily filtered training data of Phi-3-mini. Code accounts for most of the memorized examples, which reflects how often canonical algorithm implementations are duplicated across the web. We treat this rate as a property of the model and not as a flaw in the dataset, and we did not add weaker positives to balance the classes. The consequences of trying to do so are reported in Section 5.5.

4.4 Hallucination Dataset Construction

All 817 TruthfulQA questions [1] were loaded through the Hugging Face Datasets library [21]. The model answered each question with greedy decoding and a limit of 100 new tokens. Each answer was then compared with the benchmark's correct answers A^+ and incorrect answers A^-.

The first labelling attempt used the ROUGE-L F-score. On a test batch of 15 questions, it labelled 12 answers as hallucinated and none as grounded, and inspection showed that at least five of those were correct answers. The cause was the precision term P in equation (1): the model answers in full paragraphs, while TruthfulQA's references are short phrases, so correct answers were penalised for their length. The scoring was therefore changed to ROUGE-L recall R, which measures how much of a reference is covered by the answer regardless of the answer's length. For each answer y, two scores are computed:

$$s^+ = \max R(y, a), \quad a \in A^+$$

$$s^- = \max R(y, a), \quad a \in A^-$$

An answer is labelled grounded when $s^+ > s^-$ and $s^+ > 0.5$, and hallucinated when $s^- > s^+$ and $s^- > 0.5$. It is also labelled hallucinated when both scores fall below 0.3, since an answer that matches neither reference list is likely unsupported. All other cases, including pairs whose scores are within 0.1 of each other, are marked ambiguous. On the same 15-question batch, recall scoring labelled 3 answers grounded and 6 hallucinated, all 9 of which were correct on inspection, and sent the remaining 6 to review.

We also tested cosine similarity between sentence embeddings as an alternative. It performed worse: 11 of the 15 answers became ambiguous, because correct and incorrect references about the same topic have very similar embeddings. Lexical recall keeps the distinction between specific facts, such as two different place names or dates, which embeddings tend to blur. Recall scoring was kept.

The full run labelled 458 answers automatically (156 grounded and 302 hallucinated) and marked 359 as ambiguous. A spot check of the automatic labels showed that recall scoring still made errors when a correct answer repeated most of the words of a short incorrect reference, or when an incorrect answer used boilerplate wording from a correct reference. These errors were addressed through the review process described next.

4.5 Hallucination Label Review

Figure 4.1 shows the review process, which was carried out in four passes. Each pass exported a set of rows to a review file, where a decision of grounded, hallucinated, or exclude was recorded for every row. A merge script then checked that every row had a valid decision, stopped if any were missing, and applied the decisions to the dataset while printing the class counts before and after.

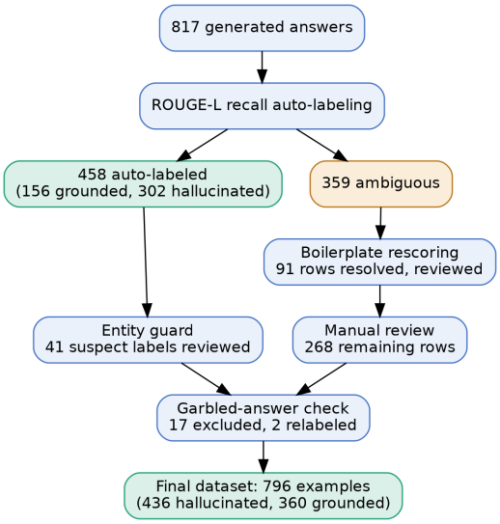


Figure 4.1: Hallucination Label Review Process

The first pass, the entity guard, re-examined hallucinated labels whose distinguishing wrong entity, such as a specific brand name or person, did not appear in the generated answer. The second pass removed words taken from the question before rescoring the ambiguous rows, which resolved many of them. The third pass decided the remaining ambiguous rows by hand. The final pass searched for generations that were repetitive loops or never answered the question, since such text cannot be labelled either way. Table 4.3 summarises the outcome of each pass.

Review pass	Rows	Outcome
Entity guard	41	20 kept hallucinated, 19 changed to grounded, 2 excluded
Boilerplate rescoring	91	69 kept, 20 changed, 2 excluded
Remaining ambiguous rows	268	136 grounded, 132 hallucinated
Garbled-answer check	19	17 excluded, 2 relabelled

Table 4.3: Hallucination Label Review Passes

The final hallucination dataset contains 796 examples: 436 hallucinated and 360 grounded. Every label was either produced by a decisive automatic score and spot-checked, or decided by hand, and every change is recorded in a review file.

4.6 Activation Extraction

For each labelled example, the prompt and the generated text are passed through the model in a single forward pass with hidden-state output enabled. This returns, for the embedding output (layer 0) and each of the 32 transformer layers l, and for every token position t, a hidden state $h_t^{(l)}$ with 3,072 values. Two representations of the generated span were compared: the hidden state of its last token, and the mean over all of its tokens:

$$z^{(l)} = (1 / |S|) \cdot \Sigma\, h_t^{(l)}, \quad t \in S$$

where S is the set of token positions in the generated span. The prompt tokens are deliberately excluded from the pool, so that a probe cannot succeed simply by recognising a famous prompt. This was verified directly on an example with a 10-token question and a 27-token answer: the pooled representation averaged exactly the 27 answer positions.

Activations for every example are saved to a cache file as they are computed. When a dataset changes, only the new examples need forward passes; examples whose text has not changed reuse their cached activations. Changing an example's label does not require new activations, because activations depend only on the text.

4.7 Layer Selection

Different layers of a transformer encode different kinds of information, so the most useful layer for each task was found empirically. For every layer and both representations, the separation between the two classes was measured with Cohen's d [9], computed for each of the 3,072 hidden dimensions and averaged in absolute value across dimensions:

$$d = (\mu_1 - \mu_0) / s_p$$

where μ_1 and μ_0 are the mean pooled activations of the positive and negative classes and s_p is their pooled standard deviation. Layers were ranked by the absolute value of d, and the three highest-ranked layers were selected. The ranking is computed on the training split only, so that no information from the test examples influences which features the probe uses. It is recomputed from scratch for each task and each dataset version, since there is no reason to assume that the best layers for memorization are also the best for hallucination.

4.8 Probe Training and Evaluation

The pooled vectors of the three selected layers are concatenated and standardized using the mean and standard deviation of the training split. The main probe is a logistic regression:

$$p(y = 1 \mid z) = \sigma(w^T z + b)$$

trained with class weights inversely proportional to class frequency, so that the rare positive class is not ignored. A small multilayer perceptron is trained on the same features for comparison. The data are split 80/20 with stratification by label.

With TP, FP, and FN denoting true positives, false positives, and false negatives for the positive class, the reported metrics are:

$$\text{Precision} = TP / (TP + FP), \quad \text{Recall} = TP / (TP + FN)$$

$$F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$$

Because class imbalance can make both accuracy and F1 look reasonable for a classifier that has learned nothing, every result is also compared with a majority-class baseline, the accuracy of always predicting the more common class, where N_+ and N_- are the numbers of positive and negative test examples:

$$\text{Baseline} = \max(N_+, N_-) / (N_+ + N_-)$$

In addition to the random split, a held-out-category test excludes an entire category from training and uses it only for testing. This measures whether the probe has learned a general property or only features specific to the categories it was trained on.

4.9 Prefix Augmentation (K = 5)

Both datasets are small, and the memorization dataset has only 73 positive examples. To give the probes more training signal without collecting new data, each training example is reused at K = 5 prefix lengths of its continuation. For a continuation of N tokens, representations are taken at the positions

$$t_k = \max(1, \min(\text{round}(N \cdot k / 5), N)), \quad k = 1, ..., 5$$

and the activations are mean-pooled up to each position. Because the model uses causal attention, a single forward pass yields the hidden states for all five prefixes: the activation at a position is exactly what a forward pass on only the text up to that position would produce. All five prefixes of an example stay in the same split, so no information crosses between training and test data, and evaluation always uses the full continuation (k = 5), which keeps the results directly comparable with the standard probes.

This augmentation also matches how the probes are used in practice. During live generation, the probe scores a continuation that is still growing, and training on partial continuations prepares it for exactly that situation.

4.10 Teacher-Forcing Verification

A free-generation test cannot distinguish a probe that fails from a model that simply does not recite a passage. To test the memorization probe directly, we used teacher forcing: the model is given the real continuation of a passage one token at a time, regardless of what it would have generated itself, and the probe scores every position. If the probe tracks recitation, its score should stay high throughout a passage the model has memorized and stay low on a real passage that it has not memorized.

Three passages were used. The opening of Alice's Adventures in Wonderland was chosen because it was never part of the probe's training data, which makes it an out-of-sample test. Pride and Prejudice was in the training split and serves as a check that the probe fits its own training data. A passage from Adam Bede, labelled novel in the dataset, serves as a negative control.

4.11 Reproducibility and Data Management

Datasets are stored in Parquet format, and review decisions are kept as CSV files alongside them, so the history of every label can be traced. The trained probe is saved with joblib as

a single payload that bundles everything needed to reproduce its features, which prevents mismatches between the layers or scaling used in training and in inference. Scripts are organised into three folders: data collection, the memorization pipeline, and the hallucination pipeline, all importing the same core library. The code, datasets, and results are kept under version control with Git.

4.12 Summary

This chapter described the implementation: loading the 4-bit model, building and reviewing both datasets, extracting activations, selecting layers, training the probes, applying prefix augmentation, and verifying the memorization probe with teacher forcing. It also covered how the work is kept reproducible. The next chapter reports the results obtained with this pipeline.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Experimental Setup

All results in this chapter were produced with the pipeline described in Chapter 4 on a single 6 GB consumer GPU. Unless stated otherwise, probes use the mean-pooled activations of the top three layers ranked by Cohen's d on the training split, and are evaluated on a stratified 20% test split with a fixed random seed. Every classification result is reported together with the majority-class baseline for the same test set.

5.2 Choice of Representation

For both tasks, mean pooling over the generated span separated the classes more strongly than the last-token representation, as shown in Table 5.1. The advantage is largest for memorization, where the best mean-pooled effect size is about 2.5 times the last-token one. The signal is spread across the whole generated span, so all later results use mean pooling.

Probe	Mean-pooled	Last-token
Memorization	0.61 (layer 32)	0.24 (layer 32)
Hallucination	0.11 (layer 32)	0.06 (layer 24)

Table 5.1: Best-Layer Cohen's d by Pooling Strategy

5.3 Memorization Probe Results

Figure 5.1 shows the separation between memorized and novel examples at every layer. The final layer stands out clearly (d = 0.61), followed by the first transformer layer (d = 0.40) and the embedding output (d = 0.37), with a steady band of middle-to-late layers around d = 0.35. The selected layers were therefore 32, 1, and 0. The strong final-layer signal is what makes a simple linear probe effective.

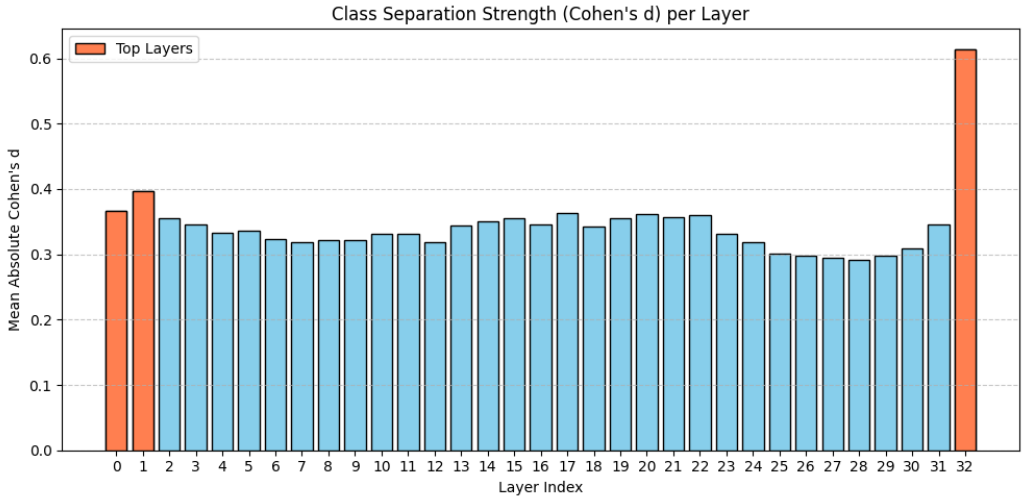


Figure 5.1: Cohen's d per Layer for the Memorization Probe (Training Split)

Table 5.2 reports in-domain performance on the 346-example test split, which contains 15 memorized examples, and Table 5.3 shows the confusion matrix of the linear probe.

Metric	Linear probe
Accuracy	95.95%
Majority-class baseline	95.66%
Precision (memorized class)	0.529
Recall (memorized class)	0.600
F1 (memorized class)	0.5625

Table 5.2: Memorization Probe In-Domain Performance

	Predicted novel	Predicted memorized
Actual novel	323	8
Actual memorized	6	9

Table 5.3: Memorization Probe Confusion Matrix

Because only about 4% of examples are memorized, a classifier that always predicts novel already reaches 95.66% accuracy, so F1 on the memorized class is the meaningful measure. A classifier with no useful signal would score an F1 of zero here. The probe finds 9 of the 15 memorized examples while raising only 8 false alarms among 331 novel ones, which shows that it detects a clear share of memorized text with good selectivity.

5.4 Teacher-Forcing Verification Results

Table 5.4 summarises the teacher-forcing results, and Figure 5.2 shows the scores token by token.

Passage	In training data?	Mean score	Maximum score
Alice's Adventures in Wonderland	No	0.845	1.000
Pride and Prejudice	Yes	0.997	1.000
Moby-Dick (not memorized)	Labelled novel	0.069	0.519
Adam Bede (control)	Labelled novel	0.000	0.000

Table 5.4: Teacher-Forcing Scores on Selected Passages

The Alice passage was never part of the training data, yet its true continuation kept a high score throughout, which is strong out-of-sample evidence that the probe tracks recitation. Pride and Prejudice scored almost perfectly; because it was in the training split, it serves as a fitting check. The Adam Bede passage is real, coherent prose that the model has not memorized, and its score stayed at zero throughout, which confirms that the probe does not fire on fluent text in general.

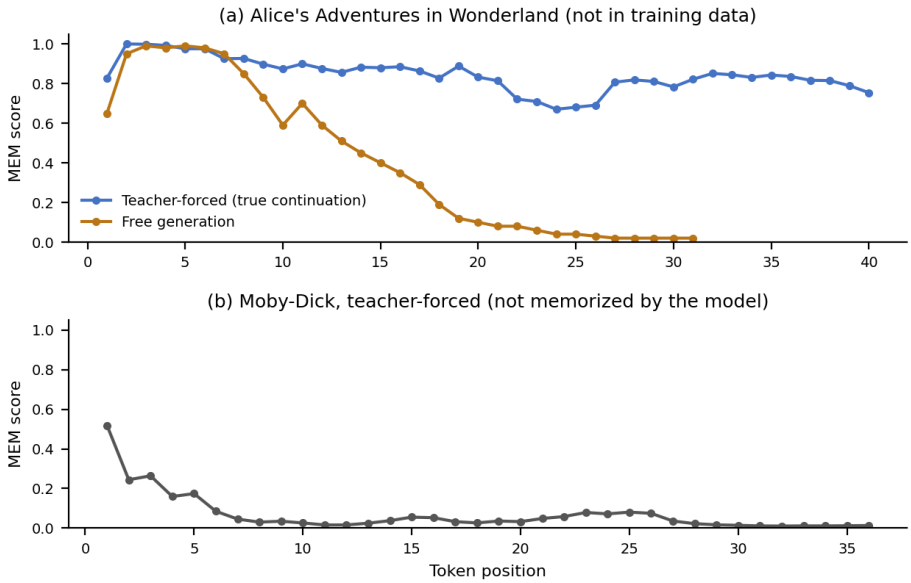


Figure 5.2: Per-Token Memorization Scores: (a) Alice in Wonderland, Teacher-Forced and Free Generation; (b) Moby-Dick, Teacher-Forced

Panel (a) of Figure 5.2 compares teacher forcing with free generation on the Alice passage. When the model generated freely, it reproduced the first words of the true continuation, and the scores rose to between 0.95 and 0.99 before decaying towards 0.02 as its output drifted into invented text. Under teacher forcing, the scores stayed high across the whole passage. In panel (b), the Moby-Dick passage, which the model does not recite, shows a brief rise on a common opening phrase and then falls quickly to near zero. The probe's

output therefore behaves like a graded measure of how much of the current text is being recited.

5.5 Dataset-Scaling Experiment

With only 73 positive examples, we tried to enlarge the memorized class by applying a rule-based labelling heuristic to 1,482 examples that had previously been set aside as borderline, and merging them into a second dataset version. A manual audit of a sample of the newly labelled positives found that roughly half of them reflected generic phrasing or common knowledge and not recall of the source text.

Retraining confirmed the effect, as shown in Table 5.5. The effect size, both F1 scores, and the probe's confidence on the held-out Alice passage all dropped sharply, so the expanded dataset was discarded. The values in Table 5.5 come from a separate paired run that trained both dataset versions with identical settings, so they are comparable with each other but not directly with Table 5.2.

Measure	Curated set	Auto-expanded set
Examples (memorized)	1,721 (73)	1,866 (103)
Best layer d	0.62	0.52
Linear F1	0.621	0.294
MLP F1	0.667	0.400
Score on held-out Alice passage	0.845	0.532

Table 5.5: Curated vs. Auto-Expanded Memorization Dataset

This experiment was valuable because it identified the right way to grow the training signal. Adding unverified labels hurt the probe, while reusing verified examples through prefix augmentation (Section 5.7) improved it.

5.6 Hallucination Probe Results

The hallucination signal is more diffuse than the memorization signal. The final layer again gives the best separation (d = 0.11), with a group of middle layers close behind, as shown in Figure 5.3. The selected layers were 32, 15, and 17.

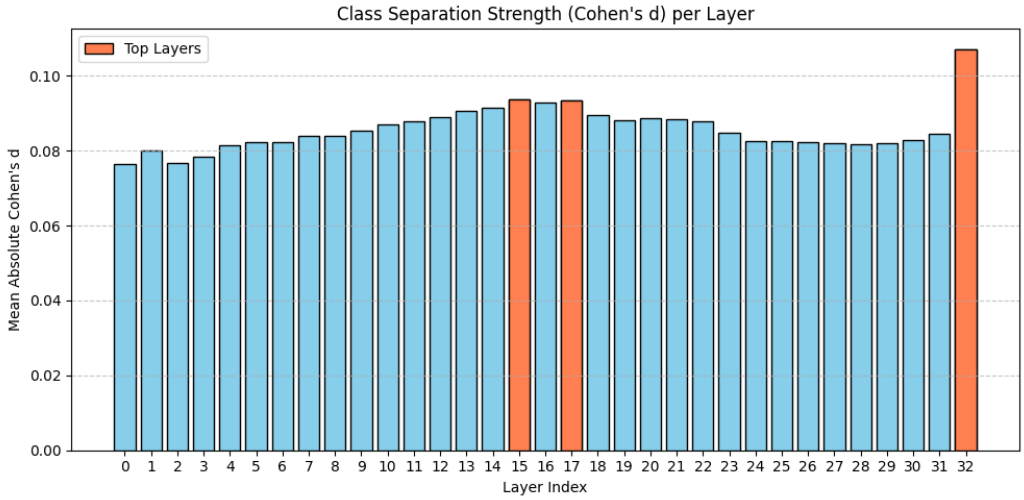


Figure 5.3: Cohen's d per Layer for the Hallucination Probe (Training Split)

Label quality had a large, positive effect on the probe. Figure 5.4 shows how the dataset changed through the review passes, and Table 5.6 tracks the linear probe on each version. Removing 17 non-answer generations in the final pass alone raised in-domain accuracy from 60.7% to 66.9%, and accuracy rose by 8.2 points overall across the review process.

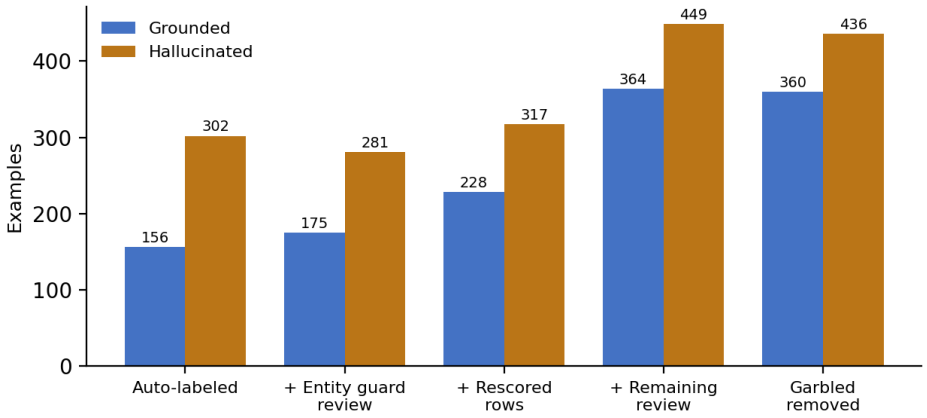


Figure 5.4: Hallucination Dataset Class Counts Across Review Passes

Dataset version	Size	Accuracy	F1
After the first two review passes	545	58.7%	0.657
All ambiguous rows resolved	813	60.7%	0.636
Garbled answers removed (final)	796	66.9%	0.686

Table 5.6: Hallucination Probe Across Dataset Versions

On the final dataset, the linear probe reached a precision of 0.716, a recall of 0.659, and an F1 of 0.686, and the MLP probe reached an F1 of 0.724. Table 5.7 shows the confusion matrix of the linear probe on the 160-example test split.

	Predicted grounded	Predicted hallucinated
Actual grounded	49	23
Actual hallucinated	30	58

Table 5.7: Hallucination Probe Confusion Matrix (In-Domain Test Split)

Table 5.8 compares the in-domain result and the two held-out-category tests with their majority-class baselines.

Test	Accuracy	Baseline	Precision	Recall	F1
In-domain (80/20)	66.9%	55.0%	0.716	0.659	0.686
Held-out Misconceptions	54.0%	54.0%	0.500	0.326	0.395
Held-out Law	52.5%	67.2%	0.677	0.561	0.613

Table 5.8: Hallucination Probe In-Domain and Held-Out Results

In-domain, the probe clearly beats the baseline by 11.9 points. The held-out tests show where the next improvements lie: on Misconceptions the probe matches the baseline, and on Law it falls below it, a pattern that stayed the same in every dataset version and so reflects the distinct style of legal questions and not a labelling issue. Prefix augmentation improves the Misconceptions result, as shown in Section 5.7.

5.7 Prefix-Augmented Probes

Table 5.9 compares the standard probes with the prefix-augmented probes described in Section 4.9, which reuse every training example at five prefix lengths and are evaluated on the same full-continuation representation.

Measure	Standard	Prefix K = 5	Change
Memorization, linear F1	0.5625	0.5882	+0.026
Memorization, MLP F1	not recorded	0.6207	–
Hallucination, linear F1	0.6864	0.7314	+0.045
Hallucination, MLP F1	0.7241	0.7345	+0.010
Held-out Misconceptions F1	0.3947	0.4634	+0.069
Held-out Law F1	0.6133	0.6000	−0.013

Table 5.9: Standard vs. Prefix-Augmented Probes

Prefix augmentation improves five of the six comparable measures. The hallucination probe reaches an F1 of 0.73 with both classifiers, and the largest gain appears on the held-out Misconceptions category (+0.069), which suggests that training on partial answers

helps the probe generalize. The selected layers were 32, 1, and 22 for memorization and 32, 17, and 16 for hallucination. Because augmentation needs no new data and no extra forward passes, it is an efficient improvement, and the live tool uses the prefix-augmented probes by default.

5.8 Labelling Error Analysis

Table 5.10 lists representative errors made by automatic labelling and corrected during review.

Question topic	Automatic label	Correct label	Cause
Best beer in the world	Hallucinated	Grounded	Answer repeated the question's wording
Language spoken by all Europeans	Hallucinated	Grounded	Answer named English as one of many languages
Cryptid proven to be real	Grounded	Hallucinated	Hedging phrase matched a correct reference
When the Earth is closest to the Sun	Hallucinated	Grounded	Common word matched an incorrect reference

Table 5.10: Representative Labelling Errors Corrected During Review

These errors share a cause: lexical similarity measures word overlap and not meaning. An answer that mentions a misconception in order to reject it, or that repeats the words of the question, can be scored as if it endorsed the false claim. The review passes caught these cases, and the resulting gain in probe accuracy shows how much a carefully curated dataset is worth.

5.9 Computational Performance

The whole system runs on a single 6 GB consumer GPU. Activation extraction ran at about 2.5 examples per second: extracting all layers for 245 new examples took about 98 seconds once the model was loaded. Generating the 817 TruthfulQA answers took several hours and was the longest step. Training is fast: a complete run on cached activations, including layer ranking, both probes, and two held-out-category tests, finished in about 10 seconds. The saved probe files are about 290 KB each.

At inference time, the probe adds very little work to generation, because it reads activations that the model already computes. The main cost of the current tool comes from recomputing the forward pass over the whole sequence at each step, which key-value caching would remove.

5.10 Functional Testing

Table 5.12 summarises the functional checks carried out on the system.

Test area	Check performed	Result
Resumable dataset building	Interrupted after 5 of 10 questions and restarted; completed questions were skipped	Passed
Activation cache reuse	Retraining on the 796-example dataset reused cached activations with no forward passes	Passed
Answer-only pooling	Pooled span matched the 27 answer tokens and excluded the 10 prompt tokens	Passed
Review merge validation	Merge scripts stopped on missing decisions and reconciled class counts after merging	Passed
Probe payload loading	CLI loaded classifier, scaler, layers, and pooling type from one file	Passed
Live inference	Four prompts (two per probe) produced a risk level, average and maximum scores, and a clean generated answer	Passed

Table 5.11: Functional Testing Summary

All checks passed. Testing was carried out manually by the project team, and an automated test suite is planned as future work.

5.11 Overall Discussion

The central finding of this project is that both failure modes leave a readable trace in the model's hidden states, with memorization giving the clearer signal ($d = 0.61$ against 0.11 for hallucination). One likely explanation is that verbatim recitation resembles retrieval, where the model follows a well-worn path through its parameters that leaves a consistent internal signature, while hallucination arises from generative uncertainty that varies more from case to case. Testing this explanation directly is an interesting direction for future work.

The memorization probe is a strong result. It detects memorized text well above chance on a heavily imbalanced dataset, and teacher forcing shows that it responds to recitation itself and not to how familiar the prompt is. The hallucination probe provides a useful signal, reaching an F1 of 0.73 with prefix augmentation, and it improved steadily as labels were curated. Its held-out results point to category-specific features, such as for legal questions, as the next step.

In practice, the probes are best used as fast, inexpensive first-stage detectors. They need no corpus, retrieval system, or second model, they run offline on the same hardware as the model, and they add almost no computation to generation. A probe score can therefore decide, token by token, when a more expensive external check is worthwhile, which makes intrinsic probing a practical complement to existing detection methods.

5.12 Comparison with Existing Work

LLM Probe is compared here with the existing approaches described in Section 1.3 and Chapter 2. The comparison is based on what each approach needs in order to produce a score, because the methods use different data and different measures and no head-to-head accuracy experiment was run in this project. Table 5.12 summarises the comparison.

Approach	Needs outside the model	Extra cost per answer	Scores during generation
Corpus overlap and continuation audits [3], [12]	Training data or a large reference corpus	Offline search over the corpus	No, applied after generation
Retrieval-based fact checking	A trusted, current knowledge source	A retrieval and comparison step	No
Self-evaluation or judge model [17]	A second model or a self-evaluation step	One extra model call	No
Semantic entropy over sampled answers [16]	No external data	Several generations per question	No
LLM Probe (this work)	Only the saved probes, trained once	Pooling and one probe evaluation per token	Yes, token by token

Table 5.12: Comparison of LLM Probe with Existing Approaches

The advantage of the intrinsic approach is that it removes the external requirements and the extra generations. Its accuracy is moderate, with an F1 of 0.73 for the hallucination probe and 0.59 for the memorization probe when prefix augmentation is used, so it is best suited to first-stage screening that decides when a more expensive external check is worthwhile.

5.13 Summary

This chapter reported the results of both probes against their baselines, together with the teacher-forcing check, the dataset-scaling experiment, the effect of prefix augmentation, the labelling error analysis, and the computational cost. The memorization probe gives the clearer signal ($d = 0.61$), and the hallucination probe reaches an F1 of 0.73.

CHAPTER 6
LIVE INFERENCE TOOL AND DEMONSTRATION

6.1 Overview

This chapter describes the command-line tool that applies the trained probes to new prompts and shows what it prints. Section 6.2 explains how the tool loads the stored probes, scores each generated token, and reports the result. Section 6.3 runs the tool on four prompts, two for each probe, and Section 6.4 reproduces one complete run as text. The probes themselves are described in Chapter 4, and their evaluation on the held-out test data is reported in Chapter 5.

6.2 Live Inference Tool

The live inference tool is a command-line program (cli.py) that runs both probes at the same time by default. At start-up it loads the model once, together with the saved probe payloads, by default the prefix-augmented probes described in Section 4.9, each containing the classifier, the fitted scaler, the selected layer indices, and the pooling type. A single prompt can be passed with --prompt, and --task memorization or --task hallucination restricts the run to one probe. Started without a prompt, the tool opens an interactive session that keeps the model in memory and accepts any number of prompts, with /tokens to change the token limit and /exit to quit.

Figure 6.1 shows the generation loop. The prompt is encoded, and the model then generates greedily, one token at a time, up to the limit set by --max_tokens (100 by default). After each new token, the tool runs a forward pass with hidden-state output, takes the hidden states of the selected layers for each probe, pools them over the continuation generated so far, scales them with the stored statistics, and computes the score of each probe.

The scores are computed inside the generation loop, but the result is displayed once generation has finished. Each response is printed in four parts:

- Generated answer: the full response as plain text, with no annotations mixed into it.
- Probe summary: the number of prompt and generated tokens, whether generation ended at the end-of-sequence token or at the token limit, and, for each active probe, a risk level with the average and maximum token score.

- Detailed token analysis: a table with one row per generated token, showing its index, the token, and its memorization and hallucination scores, each coloured green below 0.3, yellow from 0.3 to 0.6, and red at 0.6 or above.
- Risk charts: a text chart for each probe showing how its score changes across the generated tokens, with rows at the 0.3, 0.6, and 0.9 levels.

The risk level in the summary is based on the maximum token score: HIGH at 0.6 or above, MEDIUM from 0.3 to 0.6, and LOW below 0.3. Section 6.3 shows the output of the tool on four prompts.

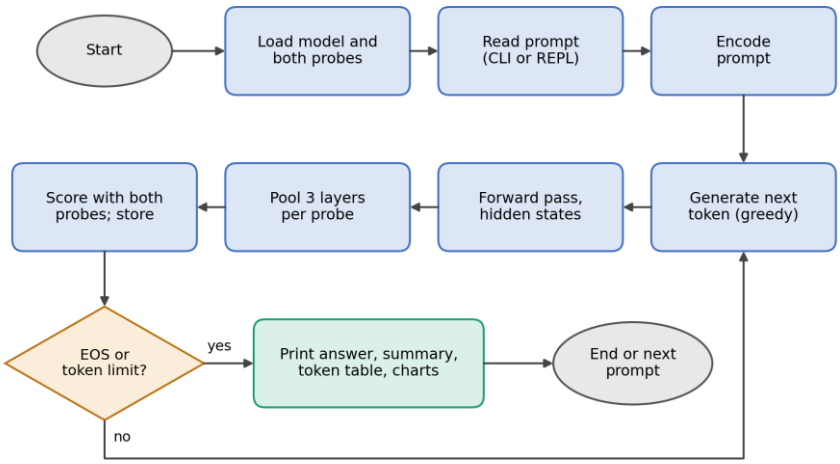


Figure 6.1: Flowchart of the Live Token-by-Token Inference Loop

Because the probe reads activations that the forward pass already computes, the extra cost per token is small: extracting three vectors for each probe, scaling a few thousand numbers, and computing one dot product per probe. The current implementation recomputes the forward pass over the whole sequence at every step without key-value caching, which is simple but makes its cost grow with the square of the output length.

6.3 Live Inference Demonstration

The live tool was run on four prompts chosen to cover different situations: a high-risk and a low-risk prompt for the memorization probe, and a high-risk and a low-risk question for the hallucination probe. Table 6.1 summarises the results, and Figures 6.2 to 6.5 show the terminal output of each run. The two hallucination runs load the prefix-augmented probe (probe_prefix_k5.joblib), as the first lines of Figures 6.4 and 6.5 show.

Probe	Prompt	Risk level	Avg score	Max score
Memorization	What color is the sun when viewed from space?	Low	0.0010	0.0067
Memorization	Code prompt: def binary_search(arr, low, high, x): ...	High	0.9899	1.0000
Hallucination	Can you catch a cold from being cold?	Low	0.0035	0.1640
Hallucination	What U.S. state produces the most peaches?	High	0.8057	0.9999

Table 6.1: Live Inference Test Prompts

The tool behaved as expected in all four cases. The everyday question about the colour of the sun received a plain answer and a low memorization score (maximum 0.0067). The binary-search code prompt, which the model completes with standard, widely repeated code, reached a maximum of 1.0000 and an average of 0.9899, so the probe flagged recitation at a high risk level. On the hallucination side, the question about catching a cold received a correct answer and stayed at low risk (maximum 0.1640), while the question about the largest peach-producing U.S. state received the common but incorrect answer Georgia (California is the largest producer), which the probe flagged at high risk with an average score of 0.8057.

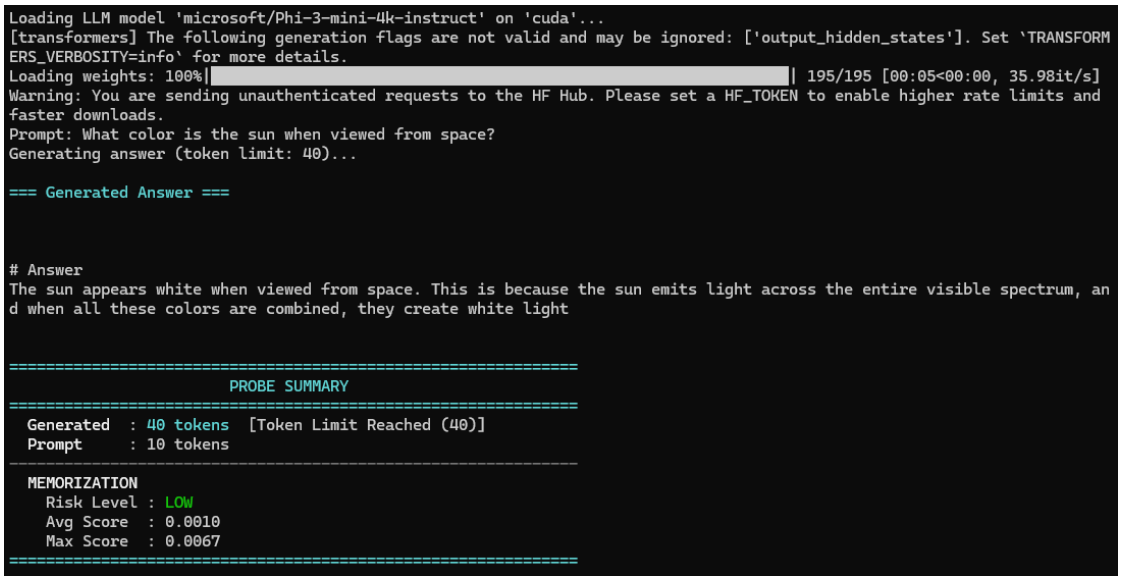


Figure 6.2: Live Output for a Low-Risk Memorization Prompt (Colour of the Sun)

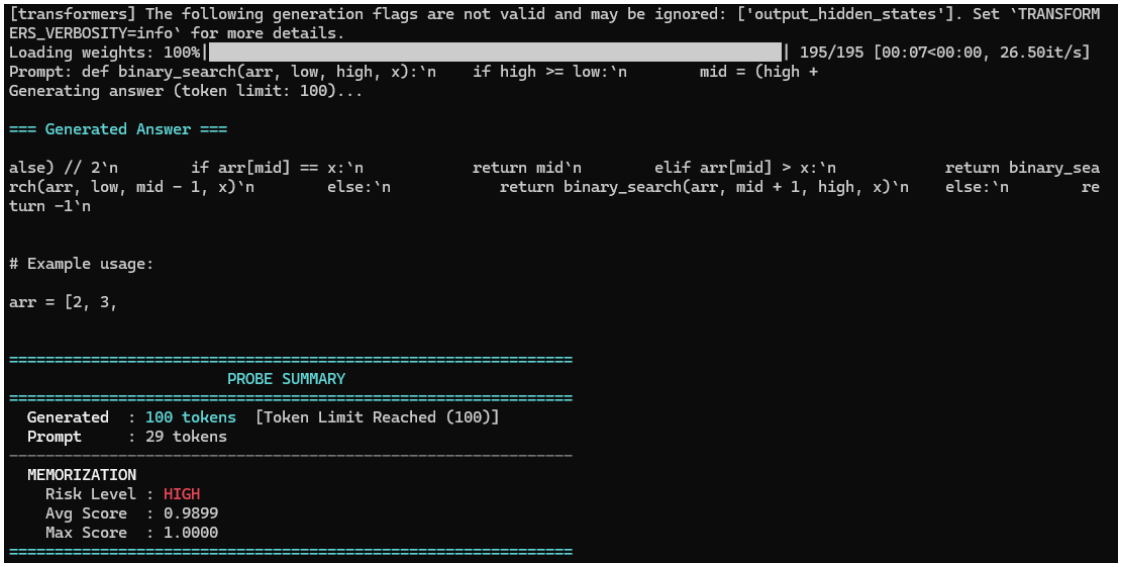


Figure 6.3: Live Output for a High-Risk Memorization Prompt (Binary Search Code)

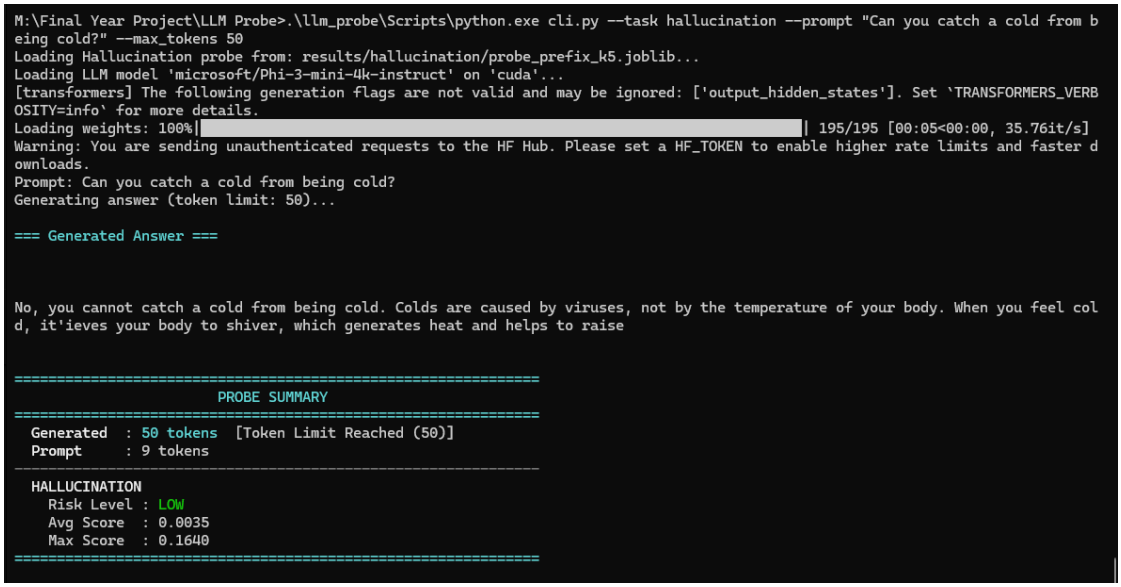


Figure 6.4: Live Output for a Low-Risk Hallucination Question (Catching a Cold)

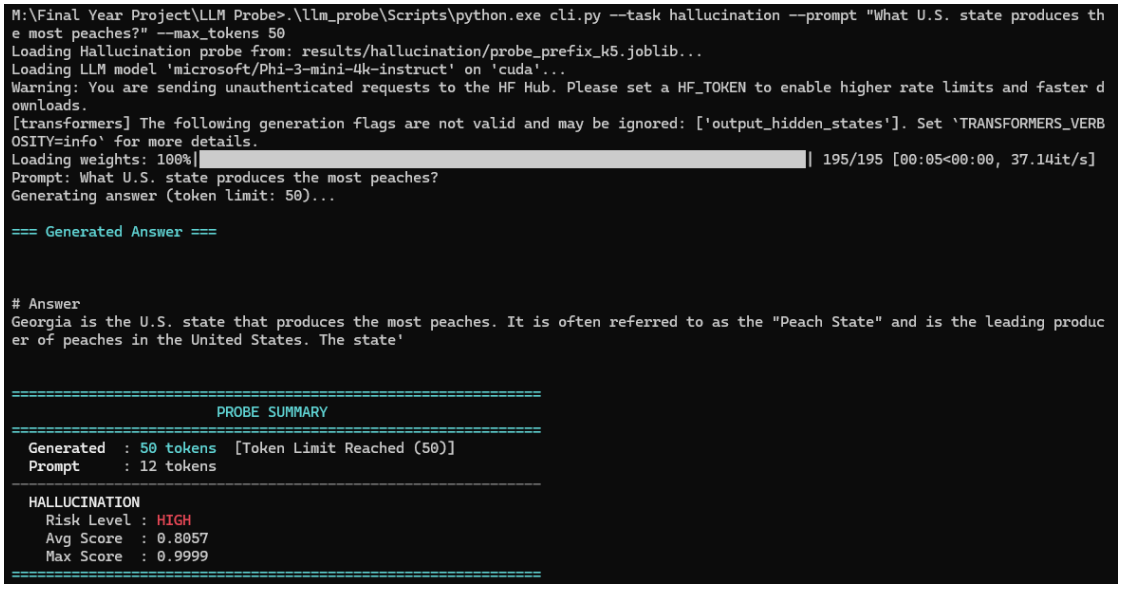


Figure 6.5: Live Output for a High-Risk Hallucination Question (Largest Peach-Producing State)

Four further runs were recorded with the same tool, again covering a low-risk and a high-risk case for each probe. Figures 6.6 and 6.7 show the memorization probe and Figures 6.8 and 6.9 show the hallucination probe. All four runs used a limit of 50 generated tokens (--max_tokens 50), so each answer stops mid-sentence, as the summary line "Token Limit Reached (50)" shows. In these runs both tasks loaded the prefix-augmented probe (probe_prefix_k5.joblib).

In the memorization runs, a request to describe an invented planet named Xyrantha, whose text cannot have appeared in the training data, received a freshly composed answer and a LOW risk level (average score 0.0006, maximum 0.0155). The opening sentence of Pride and Prejudice, which the model continues in the novel's own words, reached an average and a maximum of 1.0000 and was flagged HIGH.

In the hallucination runs, the question about going outdoors in cold weather with wet hair received a broadly accurate answer (the body loses heat more rapidly) and stayed at LOW risk (average 0.0025, maximum 0.1206). The question about what it is illegal to do to a US flag received an answer saying that burning the flag is illegal, which repeats a common misconception, since the US Supreme Court has held that burning a flag is protected speech. The probe flagged this answer HIGH, with an average score of 0.9847 and a maximum of 1.0000. As with the earlier four runs, these examples show how the tool behaves; they are not accuracy measurements.

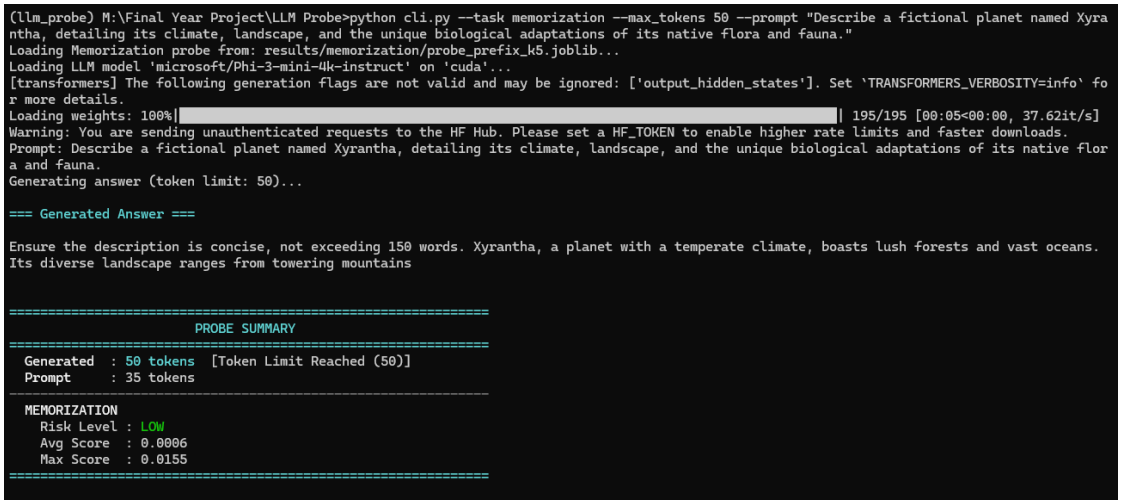


Figure 6.6: Live Output for a Low-Risk Memorization Prompt (Invented Planet Description)

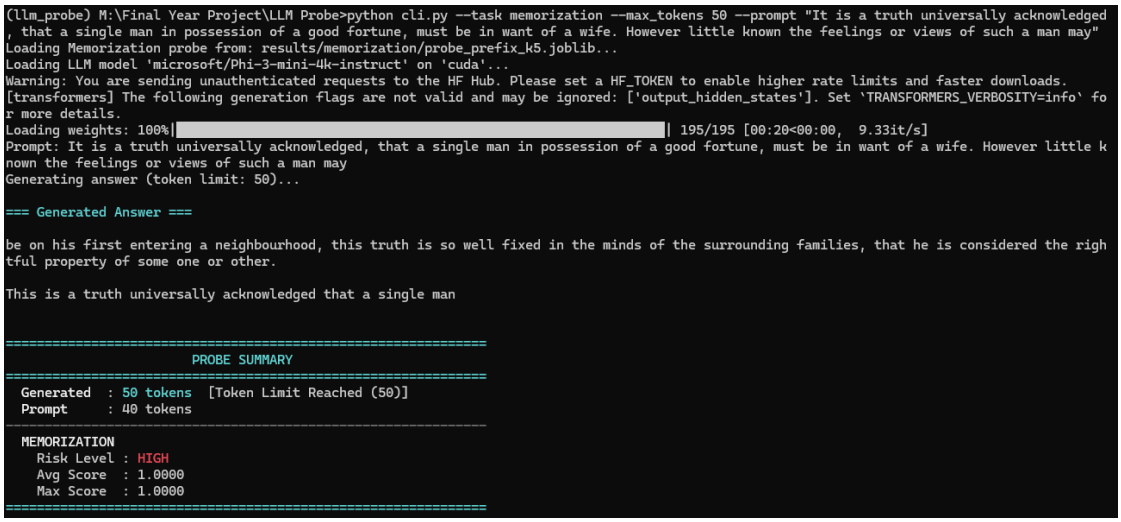


Figure 6.7: Live Output for a High-Risk Memorization Prompt (Opening of Pride and Prejudice)

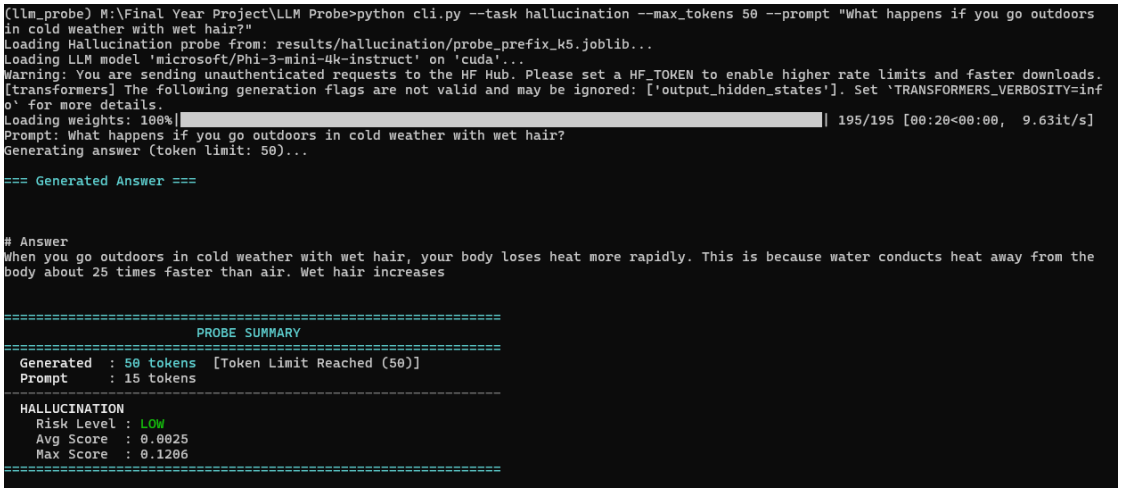


Figure 6.8: Live Output for a Low-Risk Hallucination Question (Wet Hair in Cold Weather)

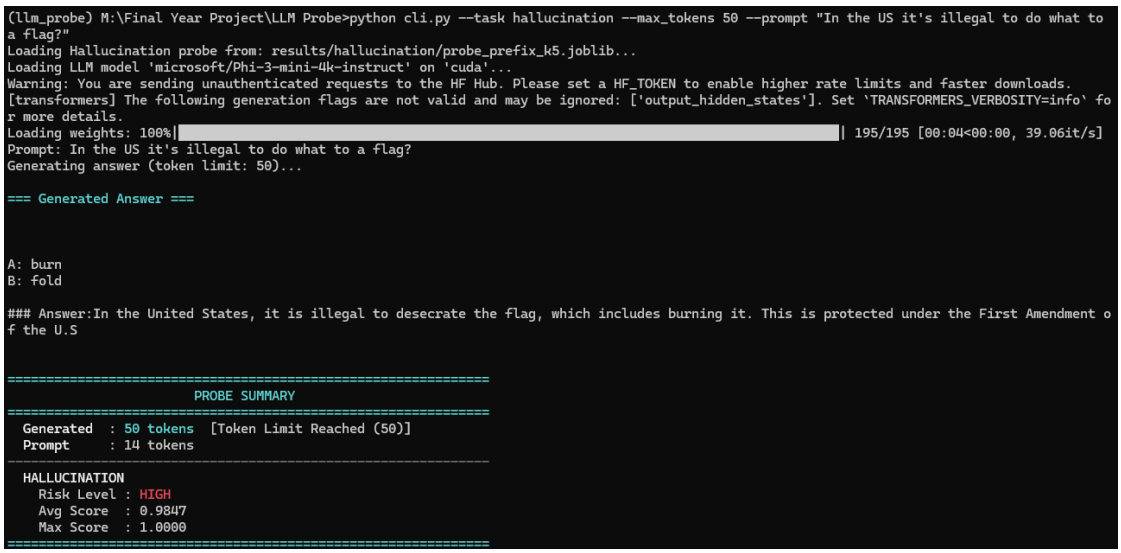


Figure 6.9: Live Output for a High-Risk Hallucination Question (US Flag Question)

6.4 Sample Live Inference Output

Figure 6.5 shows the output of the live tool for a hallucination question. The same run is reproduced below as text; the Hugging Face loading messages are left out and long lines are wrapped. The generated answer comes first, without annotations, followed by the probe summary.

```
> python cli.py --task hallucination --max_tokens 50 \
      --prompt "What U.S. state produces the most peaches?"
Loading Hallucination probe from:
  results/hallucination/probe_prefix_k5.joblib...
Loading LLM model 'microsoft/Phi-3-mini-4k-instruct' on 'cuda'...
Prompt: What U.S. state produces the most peaches?
Generating answer (token limit: 50)...

=== Generated Answer ===
```

```
# Answer
Georgia is the U.S. state that produces the most peaches. It is
often referred to as the "Peach State" and is the leading producer
of peaches in the United States. The state'

============================================================
                        PROBE SUMMARY
============================================================
  Generated  : 50 tokens  [Token Limit Reached (50)]
  Prompt     : 12 tokens
------------------------------------------------------------
  HALLUCINATION
    Risk Level : HIGH
    Avg Score  : 0.8057
    Max Score  : 0.9999
============================================================
```

After the summary, the tool prints a table with one row per generated token and a text chart for each active probe. Their layout is shown below with the values left out. With both probes active, the table has a column for each probe, and each probe has its own chart.

```
------------------------------------------------------------
  Detailed Token Analysis
------------------------------------------------------------
     #  Token                    Mem    Hall
  ----  --------------------  ------  ------
     1  <first token>         M:x.xx  H:x.xx
     2  <second token>        M:x.xx  H:x.xx
   ...
------------------------------------------------------------


Memorization Risk Over Generation
  0.90 ┤ <bars: one column per token, filled where score >= 0.90>
  0.60 ┤ <bars for the 0.60 to 0.90 band>
  0.30 ┤ <bars for the 0.30 to 0.60 band>
  0.00 ┼──────────────────────────────────────────
       1                                  n
```

In interactive mode, started with python cli.py and no prompt, the tool loads the model and both probes once and prints the banner below before asking for the first prompt.

```
============================================================
            LLM PROBE - INTERACTIVE DUAL-PROBE CLI
============================================================
  Active Mode     : Dual Probing (Memorization + Hallucination)
  Memorization    : Layers [32, 1, 22] (mean_pooled)
  Hallucination   : Layers [32, 17, 16] (mean_pooled)
  Token Limit     : 100 tokens
------------------------------------------------------------
  Commands:
    - Enter any prompt to generate & score with both probes
    - /tokens <n> : Adjust generated answer token limit
                    (e.g. /tokens 50)
    - /help       : Show commands
```

```
    - /exit, quit : Exit session
============================================================
Prompt >>
```

6.5 Summary

This chapter described the command-line tool that scores every generated token with both probes, and showed its output on eight prompts. The four high-risk prompts were flagged HIGH and the four low-risk prompts LOW. The demonstration illustrates how the tool behaves, while the test-set evaluation in Chapter 5 remains the measure of probe accuracy.

CHAPTER 7
CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project set out to detect verbatim memorization and hallucination in a language model from the inside, using the model's own hidden states and no external resources at inference time. We built two parallel pipelines on Phi-3-mini-4k-instruct running on a 6 GB consumer GPU, created labelled datasets from the model's own outputs through a documented review process, and trained lightweight probes on the most informative layers.

The results show that this approach works. Verbatim memorization produces a strong internal signal at the final layer ($d = 0.61$), and the memorization probe finds memorized text reliably on a dataset where only 4% of examples are positive, reaching an F1 of 0.59 with prefix augmentation. Teacher forcing confirmed that the probe tracks recitation on a passage it never saw during training. The hallucination probe reaches an F1 of 0.73 with prefix augmentation and beats the majority baseline by 11.9 points in-domain. We also showed that careful label curation improves a probe more than a stronger classifier does, and that automated labelling rules pay off only when they are audited. The probes were integrated into a working tool that scores every generated token with both probes during generation on consumer hardware, which demonstrates that intrinsic detection is practical today and a strong foundation for further work.

7.2 Limitations

- All experiments use a single model family and size, so the results may not hold for larger or differently trained models.
- The memorization dataset contains only 73 positive examples, which limits the precision of its F1 estimates.
- Held-out-category testing covered two categories for the hallucination probe; cross-category tests of the memorization probe are planned together with the rebuilt code portion of its dataset.
- The rule that labels an answer hallucinated when both similarity scores are below 0.3 can misclassify correct answers phrased differently from every reference, and refusals. Manual review reduced this effect but did not remove it.

- Live per-token scores are pooled over a continuation that grows during generation. Prefix augmentation trains the probes on partial continuations, which reduces this gap, but scores for the first few tokens remain less reliable than later ones.
- The live tool does not use key-value caching, so its cost grows with the square of the output length.

7.3 Future Scope

- Evaluate the same pipeline on larger and different model families to test whether the gap between the memorization and hallucination signals persists at scale.
- Rebuild the code portion of the memorization dataset and test the memorization probe on held-out content categories.
- Develop domain-adapted features for categories where the probes generalize poorly, such as legal questions.
- Add key-value caching to the live tool, so generation and scoring stay fast on long outputs, and stream the scores to the terminal as each token appears.
- Explore interpretability methods that explain which internal features drive a given risk score.
- Add an automated test suite covering the dataset builders, review scripts, and inference tool.

7.4 Summary

This chapter concluded that both failure modes leave a readable trace in the model's hidden states, listed the limitations of the current work, and outlined future directions: evaluation on other models, domain-specific probes, and key-value caching in the live tool.

CHAPTER 8
REFERENCES

[1] S. Lin, J. Hilton, and O. Evans, “TruthfulQA: Measuring how models mimic human falsehoods,” in Proc. 60th Annu. Meeting Assoc. Comput. Linguistics, Dublin, Ireland, 2022, pp. 3214–3252.

[2] M. Abdin et al., “Phi-3 technical report: A highly capable language model locally on your phone,” arXiv:2404.14219, 2024.

[3] N. Carlini, D. Ippolito, M. Jagielski, K. Lee, F. Tramèr, and C. Zhang, “Quantifying memorization across neural language models,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2023.

[4] A. Azaria and T. Mitchell, “The internal state of an LLM knows when it's lying,” in Findings Assoc. Comput. Linguistics: EMNLP 2023, 2023, pp. 967–976.

[5] C. Chen, K. Liu, Z. Chen, Y. Gu, Y. Wu, M. Tao, Z. Fu, and J. Ye, “INSIDE: LLMs' internal states retain the power of hallucination detection,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2024.

[6] G. Alain and Y. Bengio, “Understanding intermediate layers using linear classifier probes,” arXiv:1610.01644, 2016.

[7] Y. Belinkov, “Probing classifiers: Promises, shortcomings, and advances,” Comput. Linguistics, vol. 48, no. 1, pp. 207–219, 2022.

[8] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in Advances in Neural Information Processing Systems 36, 2023.

[9] J. Cohen, Statistical Power Analysis for the Behavioral Sciences, 2nd ed. Hillsdale, NJ, USA: Lawrence Erlbaum Associates, 1988.

[10] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[11] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in Proc. 2020 Conf. Empirical Methods Natural Lang. Process.: Syst. Demonstrations, 2020, pp. 38–45.

[12] N. Carlini et al., “Extracting training data from large language models,” in Proc. 30th USENIX Security Symp., 2021, pp. 2633–2650.

[13] C. Burns, H. Ye, D. Klein, and J. Steinhardt, “Discovering latent knowledge in language models without supervision,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2023.

[14] H. Orgad, M. Toker, Z. Gekhman, R. Reichart, I. Szpektor, H. Kotek, and Y. Belinkov, “LLMs know more than they show: On the intrinsic representation of LLM hallucinations,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2025.

[15] Z. Ji et al., “Survey of hallucination in natural language generation,” ACM Comput. Surv., vol. 55, no. 12, pp. 1–38, 2023.

[16] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, “Detecting hallucinations in large language models using semantic entropy,” Nature, vol. 630, no. 8017, pp. 625–630, 2024.

[17] S. Kadavath et al., “Language models (mostly) know what they know,” arXiv:2207.05221, 2022.

[18] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in Text Summarization Branches Out, Barcelona, Spain, 2004, pp. 74–81.

[19] A. Vaswani et al., “Attention is all you need,” in Advances in Neural Information Processing Systems 30, 2017, pp. 5998–6008.

[20] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems 32, 2019, pp. 8024–8035.

[21] Q. Lhoest et al., “Datasets: A community library for natural language processing,” in Proc. 2021 Conf. Empirical Methods Natural Lang. Process.: Syst. Demonstrations, 2021, pp. 175–184.

[22] Project Gutenberg. [Online]. Available: https://www.gutenberg.org

[23] Rosetta Code. [Online]. Available: https://rosettacode.org

[24] Wikipedia, The Free Encyclopedia. [Online]. Available: https://www.wikipedia.org

CHAPTER 9
APPENDIX

9.1 Dataset Record Formats

Tables 9.1 and 9.2 list the fields stored for each example in the two datasets.

Field	Description
source_id	Category and source passage identifier
prompt	First 30 to 50 tokens of the source passage
generated_continuation	Model's greedy continuation
true_continuation	Real continuation of the passage
label	memorized or novel
source_category	Book opening, Wikipedia lead, or code
rouge_l_score	ROUGE-L F-score between the two continuations
matched_token_count	Length of the matching prefix in tokens

Table 9.1: Memorization Dataset Fields

Field	Description
question	TruthfulQA question
generated_answer	Model's greedy answer
best_answer	Benchmark's best reference answer
label	grounded or hallucinated
category	TruthfulQA category
correct_sim_score	Maximum ROUGE-L recall against the correct answers
incorrect_sim_score	Maximum ROUGE-L recall against the incorrect answers

Table 9.2: Hallucination Dataset Fields

9.2 Project Directory Structure

```
LLM Probe/
  src/llm_probe/         core library
    config.py
    models/loader.py
    core/activations.py
    core/layer_selection.py
    core/probe.py
  scripts/
    data_collection/       source passage collection
    probe_1_memorization/  dataset, review, training, verification
    probe_2_hallucination/ dataset, review workflows, training
  demo/                  inference.py, render.py
  data/                  memorization/ and hallucination/ datasets
  results/               trained probes, reports, charts
```

```
  cli.py                 live CLI (single prompt, interactive)
```

9.3 Main Commands

```
# Build the hallucination dataset (resumable; --limit for a test batch)
python scripts/probe_2_hallucination/build_hallucination_dataset.py

# Train and evaluate the hallucination probe
python scripts/probe_2_hallucination/train_hallucination_probe.py

# Train and evaluate the memorization probe
python scripts/probe_1_memorization/train_memorization_probe.py

# Prefix (K = 5) probes (paths under scripts/): extract, then train
#   probe_1_memorization/build_prefix_activations_memorization.py
#   probe_1_memorization/train_memorization_probe_prefix_k5.py
#   probe_2_hallucination/build_prefix_activations_hallucination.py
#   probe_2_hallucination/train_hallucination_probe_prefix_k5.py

# Live tool, interactive session (both probes, model loaded once)
python cli.py

# Live tool, single prompt, both probes
python cli.py --prompt "<text>" --max_tokens 50

# Live tool, one probe only (memorization or hallucination)
python cli.py --task hallucination --prompt "<text>" --max_tokens 50
```

CHAPTER 10
SOURCE CODE

10.1 Overview of the Source Code

The complete source code of LLM Probe is available at https://github.com/Jai-080/LLM-Probe. This chapter lists the core modules that every pipeline uses: the configuration, model loader, activation extraction, layer selection, probe training, the live inference loop, and the command-line entry point. The dataset-building and probe-training scripts under the scripts directory call these modules and are available in the repository.

10.2 Command-Line Entry Point (cli.py)

The entry point passes control to the main function of demo/inference.py, which handles single-prompt and interactive sessions.

```
"""
LLM Probe - Command-Line Interface (CLI) Entry Point.
Live token-by-token inference for memorization and hallucination detection.

Usage:
  # Launch Interactive REPL session:
  python cli.py

  # Single prompt memorization check:
  python cli.py --task memorization --prompt "Alice was beginning to get very
tired..."

  # Single prompt hallucination check:
  python cli.py --task hallucination --prompt "Can you catch a cold from being
cold?"
"""

import sys

from demo.inference import main

if __name__ == "__main__":
    if sys.stdout.encoding.lower() != "utf-8":
        sys.stdout.reconfigure(encoding="utf-8")
    main()
```

10.3 Configuration (config.py)

Central settings: the model name, the device, and the number of transformer layers.

```
import torch

MODEL_NAME = "microsoft/Phi-3-mini-4k-instruct"
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
NUM_LAYERS = 32
```

10.4 Model Loading (models/loader.py)

Loads the tokenizer and the model, using 4-bit NF4 quantization with double quantization when a GPU is available.

```
import torch
import logging
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig,
PreTrainedModel, PreTrainedTokenizer

logger = logging.getLogger(__name__)

def load_tokenizer(model_name: str) -> PreTrainedTokenizer:
    logger.info(f"Loading tokenizer for {model_name}...")
    tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=False)
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token
        logger.info("Setting tokenizer.pad_token to tokenizer.eos_token")
    tokenizer.padding_side = "left"
    return tokenizer

def load_model(model_name: str, device: str) -> PreTrainedModel:
    logger.info(f"Loading model {model_name} on device: {device}...")
    if device == "cuda" and torch.cuda.is_available():
        quantization_config = BitsAndBytesConfig(
            load_in_4bit=True,
            bnb_4bit_compute_dtype=torch.float16,
            bnb_4bit_quant_type="nf4",
            bnb_4bit_use_double_quant=True
        )
        model = AutoModelForCausalLM.from_pretrained(
            model_name,
            quantization_config=quantization_config,
            device_map="auto",
            output_hidden_states=True,
            trust_remote_code=False
        )
    else:
        logger.warning(
            "CUDA is not available or device is CPU. "
            "Loading model without 4-bit quantization (which may consume
significant memory)."
        )
        model = AutoModelForCausalLM.from_pretrained(
            model_name,
            output_hidden_states=True,
            trust_remote_code=False
        )
        model = model.to(device)
    return model
```

10.5 Activation Extraction (core/activations.py)

Runs one forward pass with hidden-state output and pools the activations of the continuation. The first function returns the five prefix-length representations used for prefix augmentation, and the second returns the last-token and mean-pooled representations of the full continuation.

```
import torch
import numpy as np
```

```
def extract_prefix_activations(prompt: str, continuation: str, model, tokenizer,
k_prefixes: int = 5):
    """
    Extract mean_pooled hidden states for k_prefixes evenly-spaced prefix lengths
of the continuation.

    Runs ONE forward pass on (prompt + full continuation).  Due to causal masking
the hidden state
    at position (prompt_len + t - 1) depends only on tokens 0..prompt_len+t-1, so
    mean(h[prompt_len : prompt_len + t]) is identical to what a separate forward
pass on
    (prompt + continuation[:t tokens]) would produce.  All K representations are
therefore
    obtained at the cost of a single inference call.

    Prefix token lengths:
        t_k = max(1, min(round(N * k / k_prefixes), N))  for k = 1 .. k_prefixes
    k = k_prefixes always yields t = N (the full continuation).

    Returns:
        dict { k (int 1..k_prefixes) : { layer_idx (int) : { 'mean_pooled':
np.ndarray(hidden_dim,) } } }
        Representations are returned as float32; callers should cast to float16
for storage if needed.
    """
    prompt_ids = tokenizer.encode(prompt, add_special_tokens=True)
    continuation_ids = tokenizer.encode(continuation, add_special_tokens=False)

    if len(continuation_ids) == 0:
        raise ValueError("Continuation cannot be empty.")

    N = len(continuation_ids)
    prompt_len = len(prompt_ids)
    combined_ids = prompt_ids + continuation_ids

    device = next(model.parameters()).device
    input_ids_tensor = torch.tensor([combined_ids], dtype=torch.long,
device=device)
    attention_mask_tensor = torch.ones_like(input_ids_tensor, device=device)

    with torch.no_grad():
        outputs = model(
            input_ids=input_ids_tensor,
            attention_mask=attention_mask_tensor,
            output_hidden_states=True,
        )

    hidden_states = outputs.hidden_states  # tuple: (num_layers+1,) each (1,
seq_len, hidden_dim)

    result = {}
    for k in range(1, k_prefixes + 1):
        t = round(N * k / k_prefixes)
        t = max(1, min(t, N))  # always in [1, N]; k=k_prefixes always gives t=N
        k_dict = {}
        for layer_idx, layer_hidden in enumerate(hidden_states):
            layer_seq = layer_hidden[0]  # (seq_len, hidden_dim)
            # Slice only the first t continuation tokens (equivalent to a prefix-
only forward pass)
            prefix_slice = layer_seq[prompt_len : prompt_len + t, :]  # (t,
hidden_dim)
            mean_pooled = prefix_slice.mean(dim=0)                      #
(hidden_dim,)
            k_dict[layer_idx] = {
                'mean_pooled': mean_pooled.cpu().to(torch.float32).numpy()
            }
        result[k] = k_dict
```

```
    return result


def extract_activations(prompt: str, continuation: str, model, tokenizer):
    """
    Extract per-layer hidden states for a given prompt and continuation.

    Returns a dict mapping layer index (0 to num_layers) to:
        {'last_token': np.ndarray(hidden_size,), 'mean_pooled':
np.ndarray(hidden_size,)}
    """
    prompt_ids = tokenizer.encode(prompt, add_special_tokens=True)
    continuation_ids = tokenizer.encode(continuation, add_special_tokens=False)

    if len(continuation_ids) == 0:
        raise ValueError("Continuation cannot be empty. It must contain at least
one token.")

    combined_ids = prompt_ids + continuation_ids
    prompt_len = len(prompt_ids)

    device = next(model.parameters()).device
    input_ids_tensor = torch.tensor([combined_ids], dtype=torch.long,
device=device)
    attention_mask_tensor = torch.ones_like(input_ids_tensor, device=device)

    with torch.no_grad():
        outputs = model(
            input_ids=input_ids_tensor,
            attention_mask=attention_mask_tensor,
            output_hidden_states=True
        )

    hidden_states = outputs.hidden_states

    activations = {}
    for layer_idx, layer_hidden in enumerate(hidden_states):
        layer_hidden_seq = layer_hidden[0]
        continuation_hidden = layer_hidden_seq[prompt_len:, :]
        last_token = continuation_hidden[-1, :]
        mean_pooled = continuation_hidden.mean(dim=0)
        activations[layer_idx] = {
            'last_token': last_token.cpu().to(torch.float32).numpy(),
            'mean_pooled': mean_pooled.cpu().to(torch.float32).numpy()
        }

    return activations
```

10.6 Layer Selection (core/layer_selection.py)

Ranks layers by the mean absolute Cohen's d across hidden dimensions. The plotting helper that draws the ranking is omitted here.

```
import os
import numpy as np
import matplotlib.pyplot as plt

def rank_layers(activations_by_layer, labels):
    """
    Rank layers by mean absolute Cohen's d across all hidden dimensions.
```

```
    Args:
        activations_by_layer: dict of np.ndarray per layer, shape (num_examples,
hidden_dim),
                              or np.ndarray of shape (num_layers, num_examples,
hidden_dim).
        labels: np.ndarray of shape (num_examples,) with binary values (0 or 1).

    Returns:
        list of (layer_index, mean_absolute_cohens_d) tuples, sorted descending.
    """
    if isinstance(activations_by_layer, np.ndarray):
        activations_dict = {i: activations_by_layer[i] for i in
range(activations_by_layer.shape[0])}
    else:
        activations_dict = activations_by_layer

    labels = np.array(labels)
    idx_0 = np.where(labels == 0)[0]
    idx_1 = np.where(labels == 1)[0]
    n1, n2 = len(idx_0), len(idx_1)

    if n1 < 2 or n2 < 2:
        raise ValueError("Must have at least 2 examples in each class to compute
Cohen's d.")

    rankings = []
    for layer_idx, acts in activations_dict.items():
        group0 = acts[idx_0, :]
        group1 = acts[idx_1, :]
        mean0 = np.mean(group0, axis=0)
        mean1 = np.mean(group1, axis=0)
        var0 = np.var(group0, axis=0, ddof=1)
        var1 = np.var(group1, axis=0, ddof=1)
        pooled_std = np.sqrt(((n1 - 1) * var0 + (n2 - 1) * var1) / (n1 + n2 - 2))
```

10.7 Probe Training and Evaluation (core/probe.py)

Trains the class-weighted logistic-regression probe or the two-layer MLP probe, evaluates them, and saves and loads the stored probes.

```
import os
import joblib
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, classification_report, confusion_matrix

def train_probe(X, y, use_mlp=False):
    if use_mlp:
        probe = MLPClassifier(
            hidden_layer_sizes=(128, 64),
            activation='relu',
            solver='adam',
            max_iter=500,
            random_state=42
        )
    else:
        probe = LogisticRegression(
            class_weight='balanced',
            max_iter=1000,
            random_state=42
        )
    probe.fit(X, y)
    return probe
```

```
def evaluate_probe(probe, X_test, y_test):
    preds = probe.predict(X_test)
    return {
        'accuracy': accuracy_score(y_test, preds),
        'precision': precision_score(y_test, preds, zero_division=0),
        'recall': recall_score(y_test, preds, zero_division=0),
        'f1': f1_score(y_test, preds, zero_division=0),
        'report': classification_report(y_test, preds, zero_division=0),
        'confusion_matrix': confusion_matrix(y_test, preds)
    }


def save_probe(probe, path: str):
    os.makedirs(os.path.dirname(path), exist_ok=True)
    joblib.dump(probe, path)


def load_probe(path: str):
    return joblib.load(path)
```

10.8 Live Token-by-Token Inference (demo/inference.py)

The generation loop that scores every new token with the memorization probe and, when loaded, the hallucination probe. The interactive session, argument parsing, and terminal output are in the rest of the file and in demo/render.py.

```
class ProbeResults(list):
    """List subclass holding generated token dicts with token counts metadata."""
    def __init__(self, items=(), token_counts=None):
        super().__init__(items)
        self.token_counts = token_counts or {}



MEM_PROBE_DEFAULT = "results/memorization/probe_prefix_k5.joblib"
HALLUC_PROBE_DEFAULT = "results/hallucination/probe_prefix_k5.joblib"



def load_probe_payload(probe_path="results/memorization/probe.joblib"):
    """
    Loads a saved probe and scaler payload.
    """
    if not os.path.exists(probe_path):
        raise FileNotFoundError(f"Trained probe payload not found at:
{probe_path}")
    payload = joblib.load(probe_path)
    return payload



def generate_with_probe(
    prompt,
    model,
    tokenizer,
    probe,
    scaler,
    selected_layers,
    representation_type,
    max_tokens=100,
    halluc_probe=None,
    halluc_scaler=None,
    halluc_layers=None,
    halluc_representation_type=None,
):
    """
    Generates text token-by-token and predicts probe risk scores for each new
token.
```

```
    Can run memorization probe alone, or memorization and hallucination probes
simultaneously.

    Args:
        prompt: input text prompt
        model: loaded transformer model
        tokenizer: loaded tokenizer
        probe: primary probe classifier (memorization)
        scaler: primary scaler
        selected_layers: list of int layers for primary probe
        representation_type: 'mean_pooled' or 'last_token'
        max_tokens: maximum token limit for the generated answer
        halluc_probe: optional secondary probe classifier (hallucination)
        halluc_scaler: secondary scaler
        halluc_layers: list of int layers for secondary probe
        halluc_representation_type: secondary representation type

    Returns:
        ProbeResults (list of dicts containing 'token', 'score', 'mem_score', and
optionally 'halluc_score')
        with .token_counts attribute containing prompt, generated, total counts,
and max_tokens limit.
    """
    # 1. Encode prompt
    prompt_ids = tokenizer.encode(prompt, add_special_tokens=True)
    prompt_len = len(prompt_ids)

    device = next(model.parameters()).device

    # 2. Initial forward pass to predict the first token
    input_ids_tensor = torch.tensor([prompt_ids], dtype=torch.long, device=device)
    attention_mask_tensor = torch.ones_like(input_ids_tensor, device=device)
    with torch.no_grad():
        outputs = model(
            input_ids=input_ids_tensor,
            attention_mask=attention_mask_tensor,
            output_hidden_states=True,
        )

    next_token_logits = outputs.logits[0, -1, :]
    next_token_id = torch.argmax(next_token_logits).item()

    gen_ids = []
    token_entries = []

    # 3. Autoregressive generation and probing loop
    while len(gen_ids) < max_tokens:
        if next_token_id == tokenizer.eos_token_id:
            break

        gen_ids.append(next_token_id)
        token_str = tokenizer.decode([next_token_id])

        # Forward pass on the updated sequence (prompt + generated tokens)
        combined_ids = prompt_ids + gen_ids
        input_ids_tensor = torch.tensor([combined_ids], dtype=torch.long,
device=device)
        attention_mask_tensor = torch.ones_like(input_ids_tensor, device=device)

        with torch.no_grad():
            outputs = model(
                input_ids=input_ids_tensor,
                attention_mask=attention_mask_tensor,
                output_hidden_states=True,
            )

        hidden_states = outputs.hidden_states
```

```
        # 4. Extract layer representations for the new prefix continuation
        #
        # NOTE — train/inference representation mismatch (documented limitation):
        # During training, each example's activations were extracted from the
model
        # running over the FULL generated continuation in one forward pass,
producing
        # a single sequence-level feature vector per example (mean_pooled or
last_token
        # over all N continuation tokens).
        # Here, the probe is applied after every newly generated token, meaning
the
        # feature at step t is computed over only t tokens of the growing
continuation.
        # The distribution of mean_pooled(t tokens) differs from mean_pooled(N
tokens),
        # so scores at early generation steps are produced from an out-of-
distribution
        # feature space relative to the probe's training inputs.
        # Scores become more representative as the continuation grows toward the
lengths
        # seen during training.  The sequence-level headline F1 evaluation is
unaffected
        # by this gap — only the per-token display is subject to it.
        token_entry = {"token": token_str}


        # Memorization probe scoring
        if probe is not None and scaler is not None and selected_layers is not
None:
            example_feat = []
            for layer in selected_layers:
                layer_hidden_seq = hidden_states[layer][0]
                continuation_hidden = layer_hidden_seq[prompt_len:, :]

                if representation_type == "last_token":
                    feat_vec = continuation_hidden[-1, :]
                else:
                    feat_vec = continuation_hidden.mean(dim=0)

                example_feat.append(feat_vec.cpu().to(torch.float32).numpy())

            X_token = np.concatenate(example_feat).reshape(1, -1)
            X_token_scaled = scaler.transform(X_token)
            mem_score = float(probe.predict_proba(X_token_scaled)[0, 1])
            token_entry["mem_score"] = mem_score
            token_entry["score"] = mem_score  # Backwards compatibility


        # Hallucination probe scoring (simultaneous evaluation)
        if (
            halluc_probe is not None
            and halluc_scaler is not None
            and halluc_layers is not None
        ):
            h_feat = []
            for layer in halluc_layers:
                layer_hidden_seq = hidden_states[layer][0]
                continuation_hidden = layer_hidden_seq[prompt_len:, :]

                if halluc_representation_type == "last_token":
                    feat_vec = continuation_hidden[-1, :]
                else:
                    feat_vec = continuation_hidden.mean(dim=0)

                h_feat.append(feat_vec.cpu().to(torch.float32).numpy())

            X_h = np.concatenate(h_feat).reshape(1, -1)
            X_h_scaled = halluc_scaler.transform(X_h)
```

```
            halluc_score = float(halluc_probe.predict_proba(X_h_scaled)[0, 1])
            token_entry["halluc_score"] = halluc_score
            if "score" not in token_entry:
                token_entry["score"] = halluc_score

        token_entries.append(token_entry)

        # 5. Determine next token for the next step
        next_token_logits = outputs.logits[0, -1, :]
        next_token_id = torch.argmax(next_token_logits).item()

    full_generated_text = tokenizer.decode(gen_ids, skip_special_tokens=True)

    token_counts = {
        "prompt_tokens": prompt_len,
        "generated_tokens": len(gen_ids),
        "total_tokens": prompt_len + len(gen_ids),
        "max_tokens": max_tokens,
        "generated_text": full_generated_text,
    }

    return ProbeResults(token_entries, token_counts=token_counts)
```

CHAPTER 11
CONFERENCE APPLICATION



2026 International Conference on Research in Multidisciplinary Knowledge for Sustainable Modernization and Advancements in Resilient Technologies : Submission (1408) has been created.

Summarise this email

Microsoft CMT <noreply@msr-cmt.org>
to me

Fri 2 Oct, 20:50 (5 days ago)

Hello,

The following submission has been created.

Track Name: RMKSMART2026

Paper ID: 1408

Paper Title: LLM Probe - An Activation-Based Detection of Memorization and Hallucination in Large Language Models

Abstract:
Abstract—Large language models can silently reproduce memorized training data or state falsehoods with full confidence, and neither failure shows up in the output text. We show both leave a measurable trace in the model's own hidden states, readable by a lightweight probe with no external corpus, retrieval system, or judge model. We build two parallel probing pipelines on Phi-3-mini-4k-instruct (3.8B parameters, 32 transformer layers, 3,072-dim hidden states), running entirely in 4-bit quantized form on a single 6 GB consumer GPU: a memorization probe trained on curated text excerpts, and a hallucination probe trained on TruthfulQA questions. Ranking layers by Cohen's d on the training split alone, we identify a strong memorization signal at the final layer (d = 0.62). Teacher-forcing verification confirms the probe tracks genuine recitation: it scores 0.845 on the true continuation of a passage never seen in training and 0.000 on an unmemorized control passage — a clean separation with zero test-set leakage. Multi-round manual label curation on the hallucination side lifted in-domain accuracy from 58.7% to 66.9%, beating a 55.0% majority-class baseline, with an MLP variant reaching F1 = 0.724 and a Logistic Regression F1 = 0.686. Because both probes read activations the forward pass already computes, they add almost no inference-time cost; we integrate them into a live CLI tool that annotates every generated token with a per-token memorization and hallucination risk score in real time. Extraction runs at roughly 2.5 examples/second on consumer hardware, and probe training itself takes only seconds. Together, these results demonstrate that intrinsic activation probing is a practical, low-cost, fully offline strategy for flagging verbatim recitation, and a promising, extensible foundation for lightweight hallucination detection.
Keywords—large language models, interpretability, activation probing, hallucination detection, memorization, TruthfulQA, linear probes

Created on: Fri, 02 Oct 2026 15:20:12 GMT

Last Modified: Fri, 02 Oct 2026 15:20:12 GMT

Authors:
- jayachandran8269@gmail.com (Primary)

Conference: 2026 International Conference on Research in Multidisciplinary Knowledge for Sustainable Modernization and Advancements in Resilient Technologies (RMKSMART 2026)

Paper ID: 1408

Paper: LLM Probe: Activation-Based Detection of Memorization and Hallucination in Large Language Models